



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Aula 4

Diagrama de Classes

Disciplina: ANÁLISE E PROJETO DE SISTEMAS

Prof. Emanuel Coutinho

Agenda



- Introdução
- Diagrama de classes
- Técnicas para identificação de classes
- Construção do modelo de classes
- Modelo de classes no processo de desenvolvimento

Objetivo da Modelagem de Classes

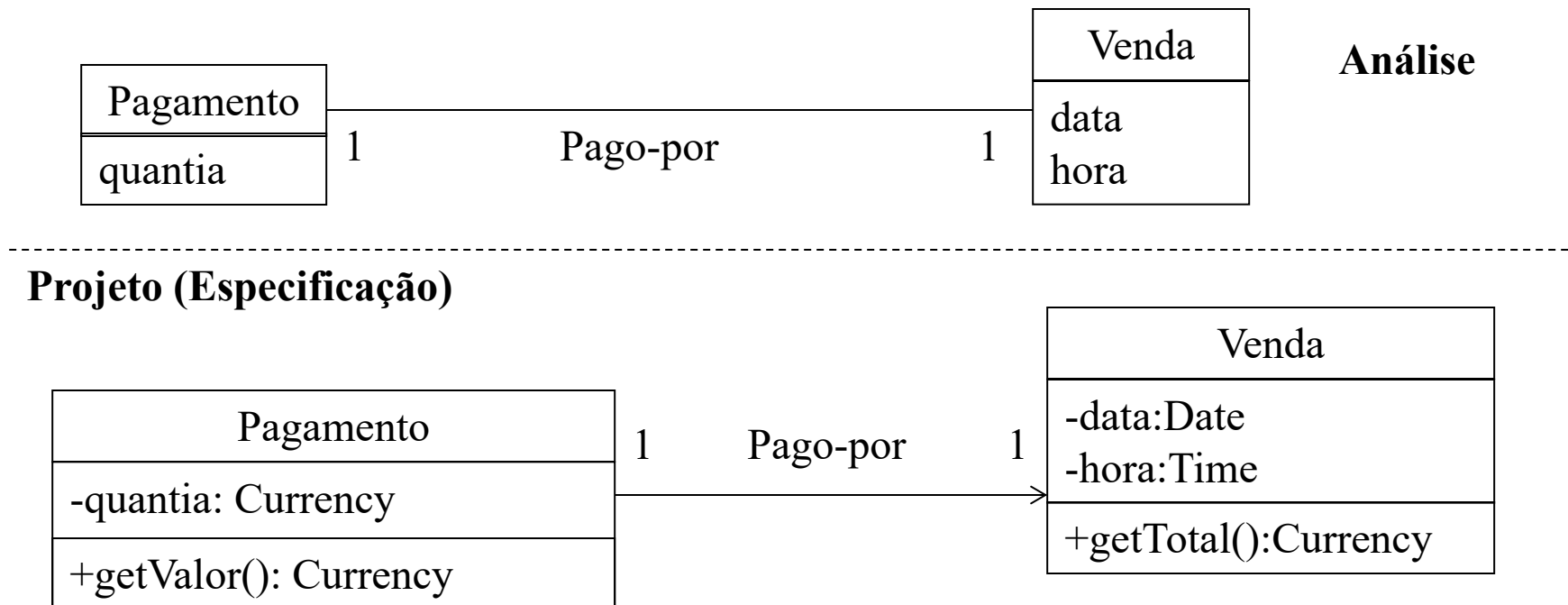
- Prover respostas para as seguintes perguntas:
 - Por definição um sistema OO é composto de objetos
 - Em um nível alto de abstração, que objetos constituem o sistema em questão?
 - Quais são as classes candidatas?
 - Como as classes do sistema estão relacionadas entre si?
 - Quais são as responsabilidades de cada classe?

Modelo de Classes de Análise

- Representa termos do domínio do negócio
 - ideias, coisas, e conceitos no mundo real
- **Objetivo**: descrever o problema representado pelo sistema a ser desenvolvido, sem considerar características da solução a ser utilizada
- Duas etapas:
 - modelo conceitual (modelo de domínio)
 - modelo da aplicação
- Elementos de notação do diagrama de classes normalmente usados na construção do modelo de análise:
 - classes e atributos, associações, composições e agregações, classes de associação, generalizações

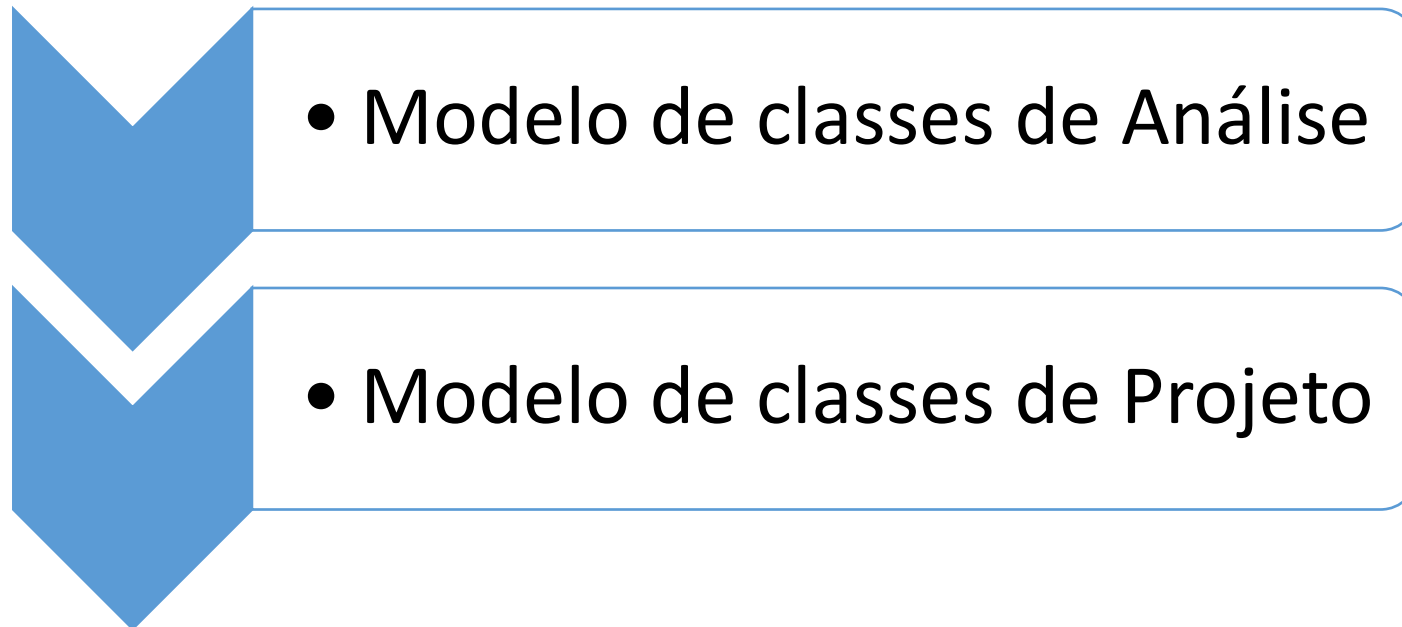
Modelo de Análise: Foco no Problema

- Não representa detalhes da solução do problema
 - Serve de ponto de partida para uma posterior definição das classe



Modelo de Classes

- O diagrama da UML utilizado para representar o aspecto estático é o diagrama de classes
- Detalhado ao longo do desenvolvimento



Classe

- Uma classe descreve objetos através de atributos e operações
 - Atributos correspondem às informações que um objeto armazena
 - Operações correspondem às ações que um objeto sabe realizar
- Notação na UML: “caixa” com no máximo três compartimentos exibidos
 - Detalhamento utilizado depende do estágio de desenvolvimento e do nível de abstração desejado

Nome da Classe

Nome da Classe
lista de atributos

Nome da Classe
lista de operações

Nome da Classe
lista de atributos
lista de operações

Exemplo (classe ContaBancária)

ContaBancária

ContaBancária
número saldo dataAbertura

ContaBancária
criar() bloquear() desbloquear() creditar() debitar()

ContaBancária
número saldo dataAbertura
criar() bloquear() desbloquear() creditar() debitar()

ContaBancária
-número : String -saldo : Quantia -dataAbertura : Date
+criar() +bloquear() +desbloquear() +creditar(in valor : Quantia) +debitar(in valor : Quantia)

Todos os objetos são instâncias de uma classe

Um objeto é a materialização de um conceito formalizado

Exemplo:

ContaEmBanco é uma classe

A conta do Donald e a tio Patinhas são instâncias



CLASSE

Carro
quantidade de pneus quantidade de portas cor
acelerar() freiar()

**INSTÂNCIA DA CLASSE
(OBJETO)**

Identidade:

O carro do João

Características:

4 pneus

4 portas

vermelho

Comportamento:

acelera

freia



CLASSE

Carro
quantidade de pneus quantidade de portas cor
acelerar() freiar()

**INSTÂNCIA DA CLASSE
(OBJETO)**

Identidade:

O carro da Maria

Características:

4 pneus

2 portas

azul

Comportamento:

acelera

freia

Refinamento de Atributos e Métodos

- Atributos e métodos de uma classe a habilitam a cumprir com suas responsabilidades
 - Atributos: permitem que uma classe armazene informações necessárias à realização de suas tarefas

[/] [visibilidade] nome [multiplicidade] [: tipo] [= valor-inicial]

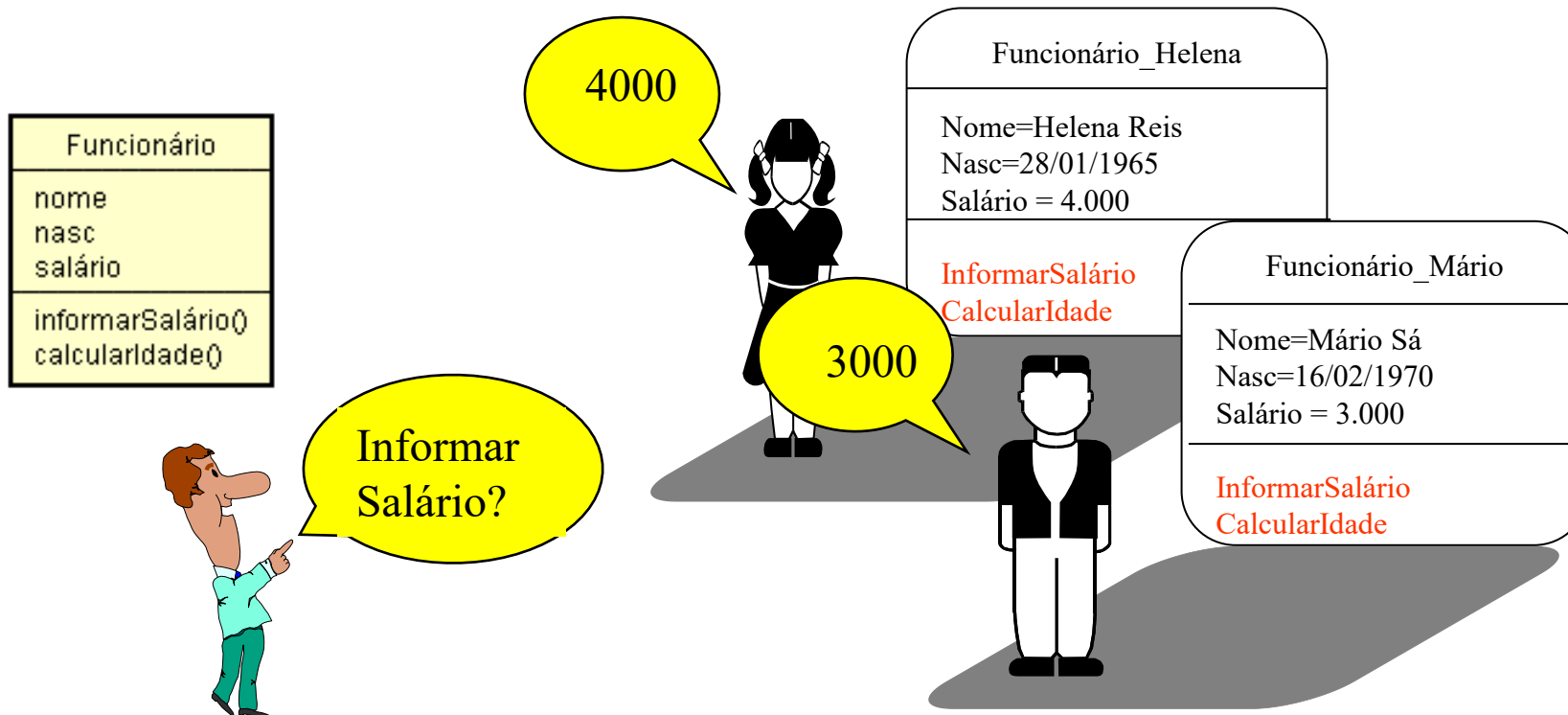
- Métodos: funções que manipulam os valores do atributos, com o objetivo de atender às mensagens que o objeto recebe

[visibilidade] nome [(parâmetros)] [: tipo-retorno] [{propriedades}]

Métodos

- Representam o comportamento das instâncias de uma classe
- Correspondem ao protocolo ou ações das instâncias de uma classe
- Um método é a implementação de uma operação
- Métodos possuem:
 - Nome
 - Argumentos
 - Variáveis locais
 - Valor de retorno

Atributos e Métodos



Visibilidade

Visibilidade	Símbolo	Significado
Pública	+	Qualquer objeto externo pode obter acesso ao elemento, desde que tenha uma referência para o objeto
Protegida	#	O elemento protegido é visível somente para objetos das subclasses da classe em que foi definido
Privativa	-	O elemento privativo é invisível externamente à classe em que este está definido
Pacote	~	O elemento é visível para qualquer instância de classes que pertençam ao mesmo pacote no qual a classe proprietária do elemento está definida

Exemplo (classe ContaBancária)

ContaBancaria

ContaBancaria

+ numero : int
+ saldo : double
+ dtAbertura : Date

ContaBancaria

+ creditar(valor : double) : boolean
+ debitar(valor : double) : boolean
+ OberSaldo() : double
+ aplicarJuros() : void

ContaBancaria

+ numero : int
+ saldo : double
+ dtAbertura : Date

+ creditar(valor : double) : boolean
+ debitar(valor : double) : boolean
+ OberSaldo() : double
+ aplicarJuros() : void

Métodos

- Alguns métodos especiais:
 - Construtores
 - Criam objetos de uma classe
 - Destrutores
 - Destroem objetos de uma classe

```
class Pessoa:
    # Método construtor: inicializa cada nova instância da classe Pessoa
    def __init__(self, nome, idade):
        self.nome = nome # Atributo para armazenar o nome
        self.idade = idade # Atributo para armazenar a idade

    # Método de instância: exibe uma mensagem de apresentação
    def apresentar(self):
        print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")

# Criação de uma instância da classe Pessoa
pessoa1 = Pessoa("João", 25)

# Chamada ao método que apresenta a pessoa
pessoa1.apresentar()
```

```

1  public class Carro{
2
3  private String cor;
4  private double preco;
5  private String modelo;
6
7  /* CONSTRUTOR PADRÃO */
8  public Carro(){
9
10 }
11
12 /* CONSTRUTOR COM 2 PARÂMETROS */
13 public Carro(String modelo, double preco){
14 //Se for escolhido o construtor sem a COR do veículo
15 // definimos a cor padrão como sendo PRETA
16 this.cor = "PRETA";
17 this.modelo = modelo;
18 this.preco = preco;
19 }
20
21 /* CONSTRUTOR COM 3 PARÂMETROS */
22 public Carro(String cor, String modelo, double preco){
23 this.cor = cor;
24 this.modelo = modelo;
25 this.preco = preco;
26 }
27
28 }

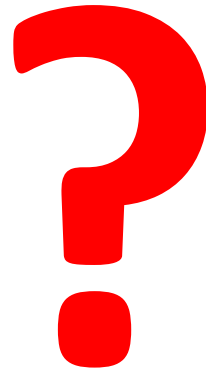
```

```

29
30 public class Aplicacao {
31
32
33 public static void main(String[] args) {
34 //Construtor sem parâmetros
35 Carro prototipoDeCarro = new Carro();
36
37 //Construtor com 2 parâmetros
38 Carro civicPreto = new Carro("New Civic", "40000");
39
40 //Construtor com 3 parâmetros
41 Carro golfAmarelo = new Carro("Golf", "38000");
42 }
43
44 }

```

Dúvidas



Exercícios



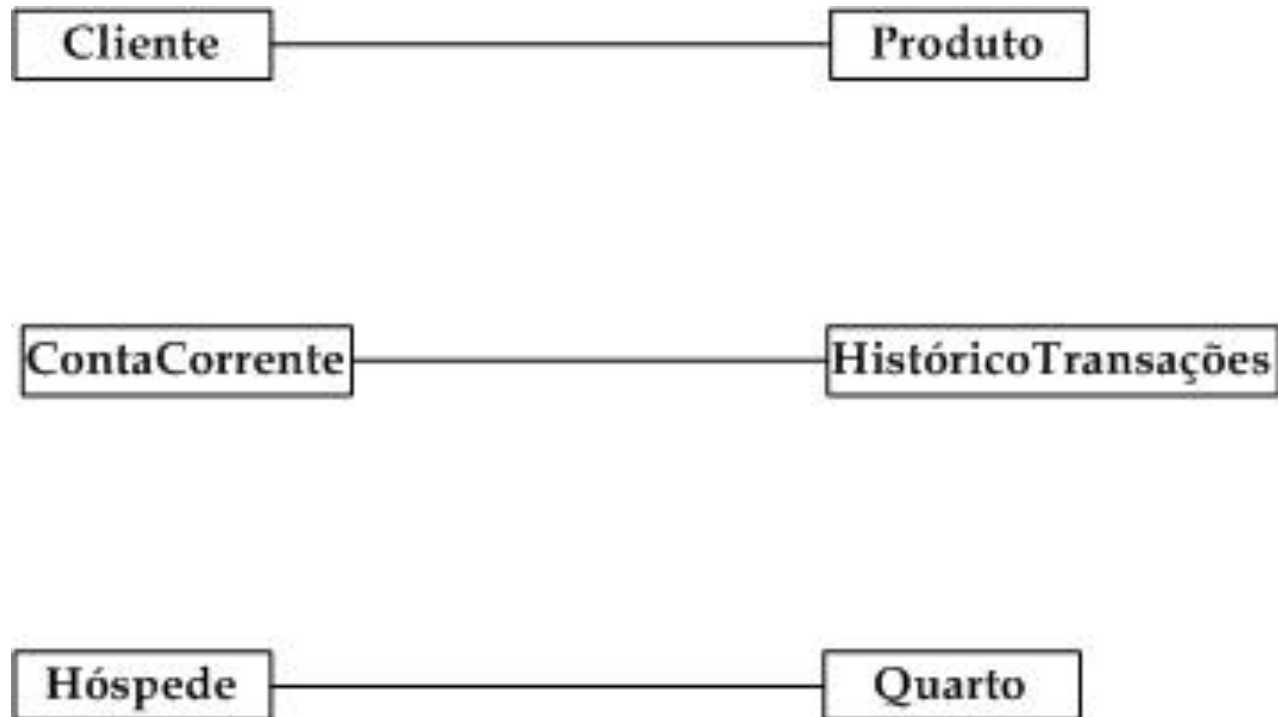
- Utilizando uma ferramenta de modelagem UML:
 - Modele classes de um sistema de controle acadêmico
 - Defina atributos
 - Defina comportamentos

Associações

- Para representar o fato de que objetos podem se relacionar uns com os outros, utilizamos associações
- Uma associação representa relacionamentos (ligações) que são formados entre objetos durante a execução do sistema
- Embora as associações sejam representadas entre classes do diagrama, tais associações representam ligações possíveis entre os objetos das classes envolvidas

Notação para Associações

- Na UML associações são representadas por uma linha que liga as classes cujos objetos se relacionam
- Ex:



Multiplicidades

- Representam a informação dos limites inferior e superior da quantidade de objetos aos quais outro objeto pode se associar
- Cada associação em um diagrama de classes possui duas multiplicidades, uma em cada extremo da linha de associação

Nome	Simbologia na UML
Apenas Um	1..1 (ou 1)
Zero ou Muitos	0..* (ou *)
Um ou Muitos	1..*
Zero ou Um	0..1
Intervalo Específico	$l_i..l_s$

Exemplos (multiplicidade)

- Ex:

- Pode haver um cliente que esteja associado a vários pedidos
- Pode haver um cliente que não esteja associado a pedido algum
- Um pedido está associado a um, e somente um, cliente



- Ex:

- Uma corrida está associada a, no mínimo, dois velocistas
- Uma corrida está associada a, no máximo, seis velocistas
- Um velocista pode estar associado a várias corridas



Associações

- Vínculo entre duas classes
- Ligação entre as instâncias de uma classe

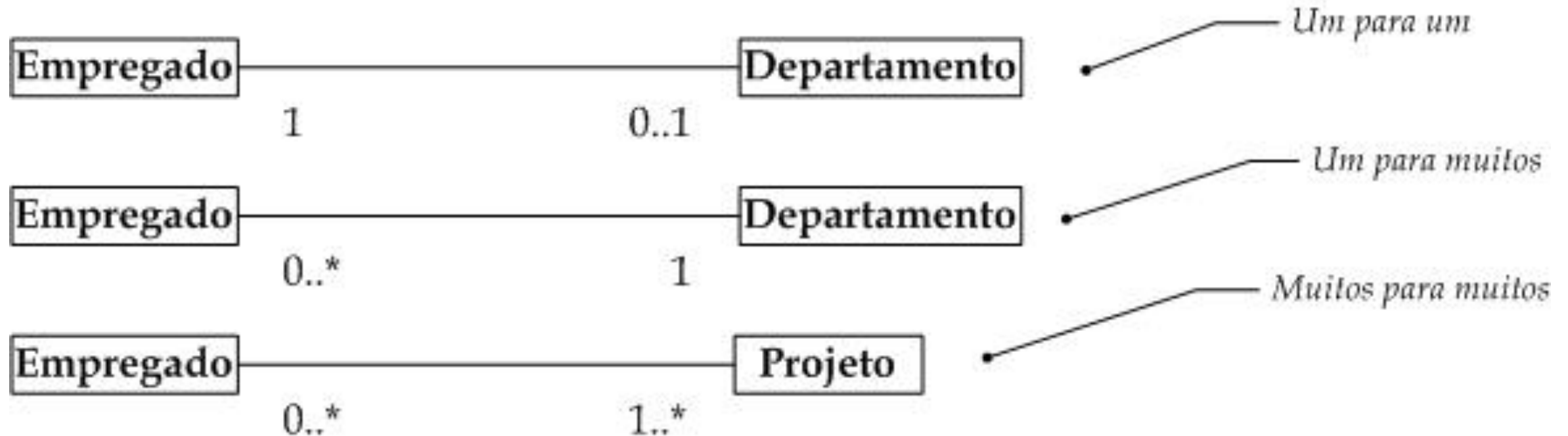


Conectividade

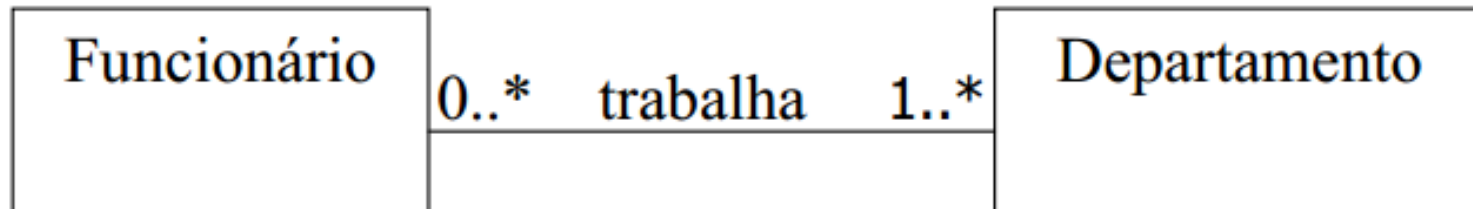
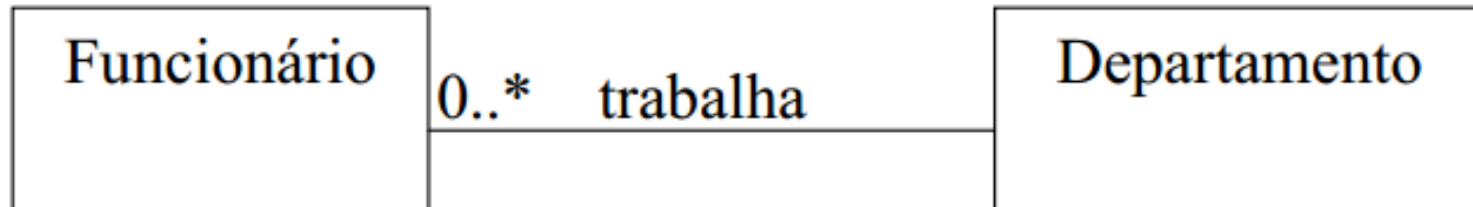
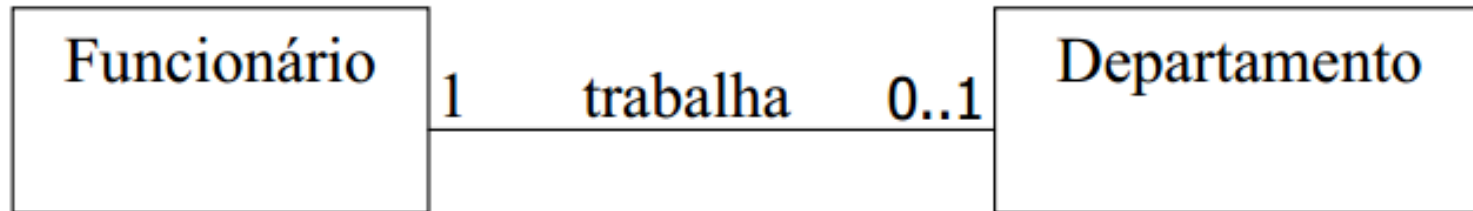
- A conectividade corresponde ao tipo de associação entre duas classes: “muitos para muitos”, “um para muitos” e “um para um”
- A conectividade da associação entre duas classes depende dos símbolos de multiplicidade que são utilizados na associação

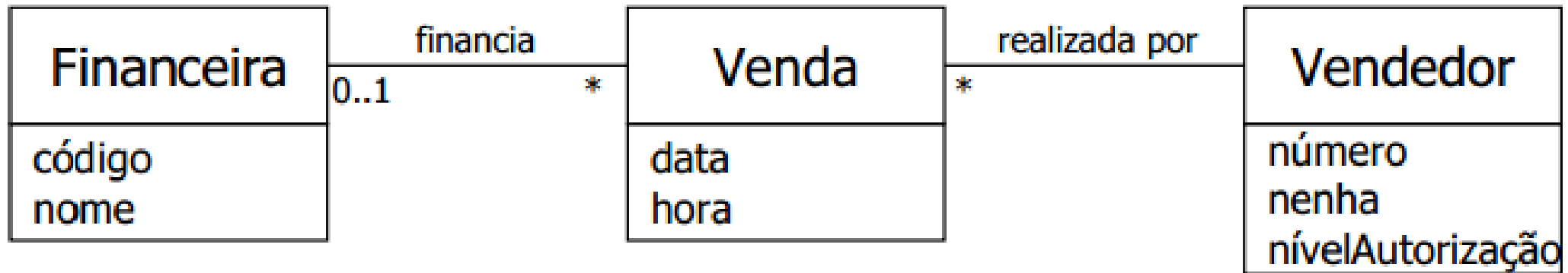
Conectividade	Em um extremo	No outro extremo
Um para um	0..1 1	0..1 1
Um para muitos	0..1 1	* 1..* 0..*
Muitos para muitos	* 1..* 0..*	* 1..* 0..*

Exemplo (conectividade)



Qual o correto?





Como ler a cardinalidade?

0..1 FINANCEIRAS financiam quaisquer VENDAS
quaisquer VENDAS são financiadas por 0..1 FINANCEIRAS

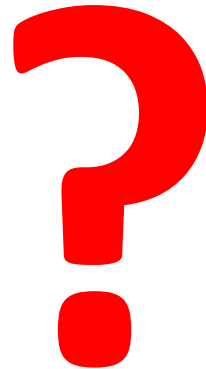
quaisquer VENDAS são realizadas quaisquer VENDEDORES
quaisquer VENDEDORES realizam quaisquer VENDAS

* = maior ou igual a zero

Participação

- Uma característica de uma associação que indica a necessidade (ou não) da existência desta associação entre objetos
- A participação pode ser obrigatória ou opcional
 - Se o valor mínimo da multiplicidade de uma associação é igual a 1 (um), significa que a participação é obrigatória
 - Caso contrário, a participação é opcional

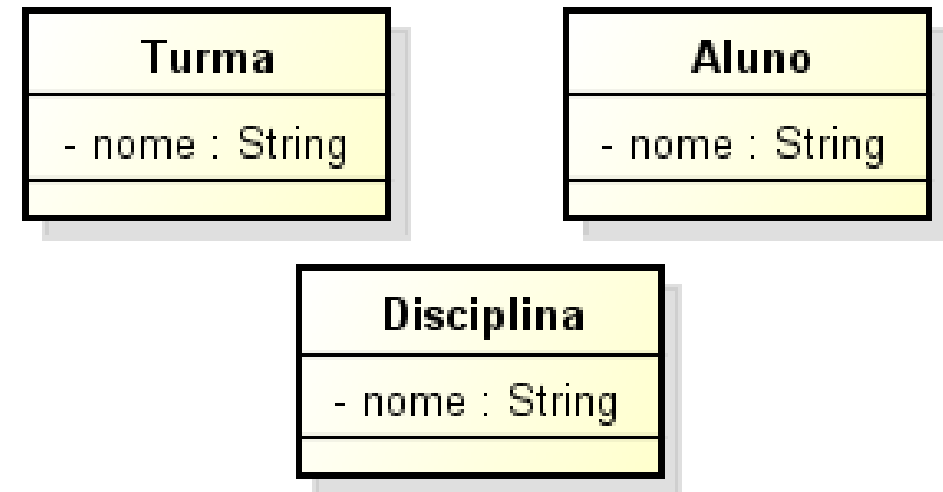
Dúvidas





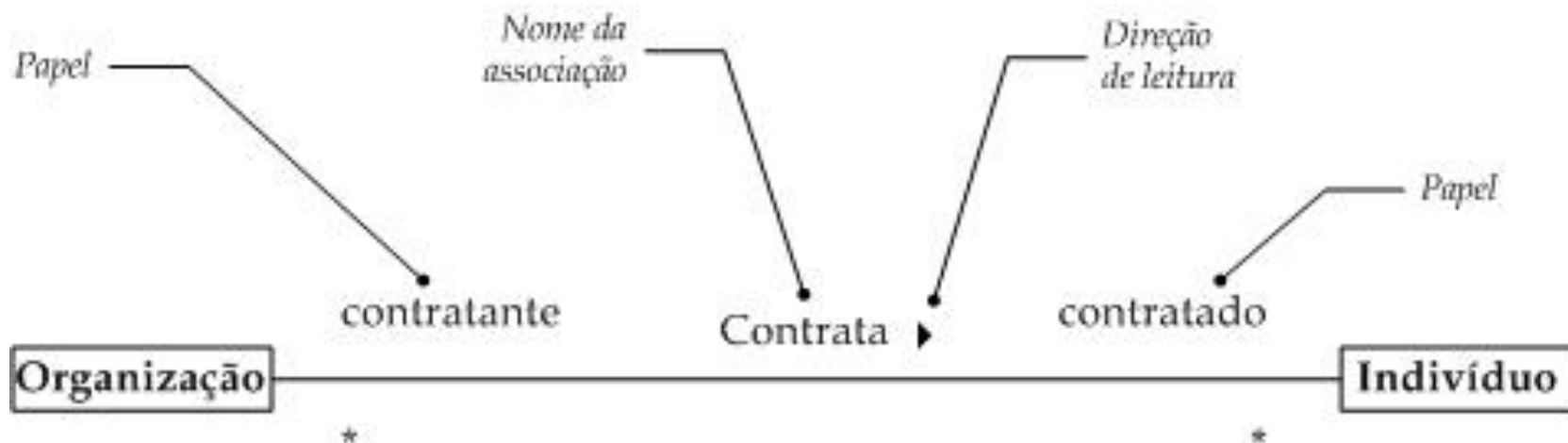
Exercícios

- Considere as 3 classes
- Contexto de um sistema de controle acadêmico
- Indique os relacionamentos entre elas e a cardinalidade



Acessórios para Associações

- Para melhor esclarecer o significado de uma associação no diagrama de classes, a UML define três recursos de notação:
 - Nome da associação: fornece algum significado semântico a mesma
 - Direção de leitura: indica como a associação deve ser lida
 - Papel: para representar um papel específico em uma associação

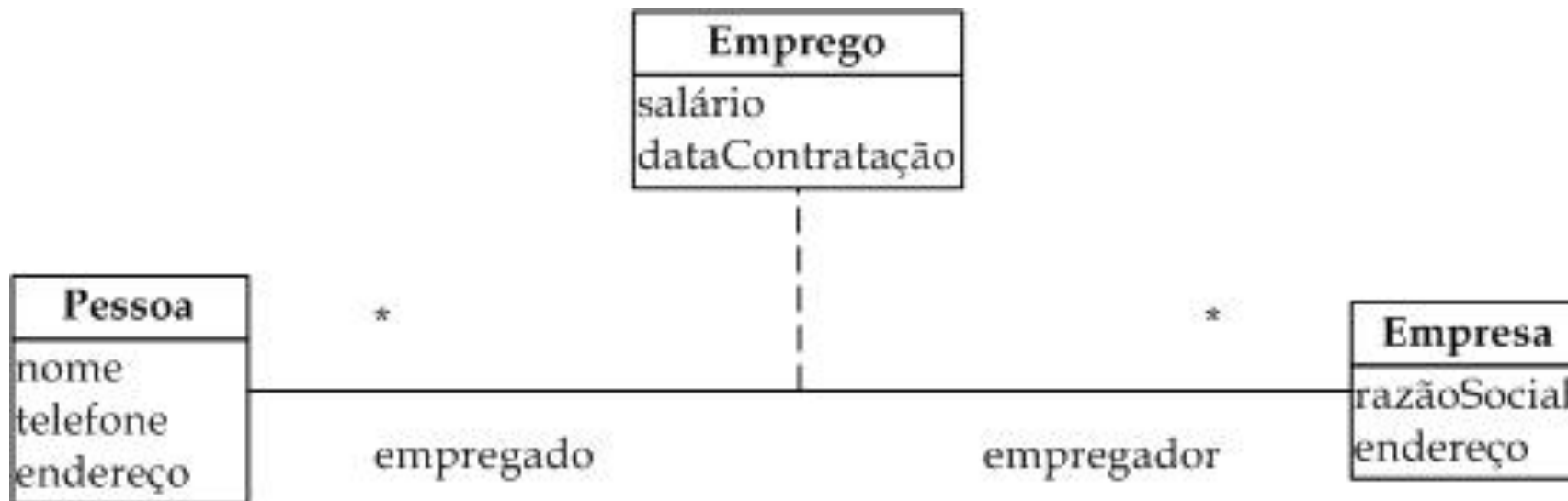


Classe associativa

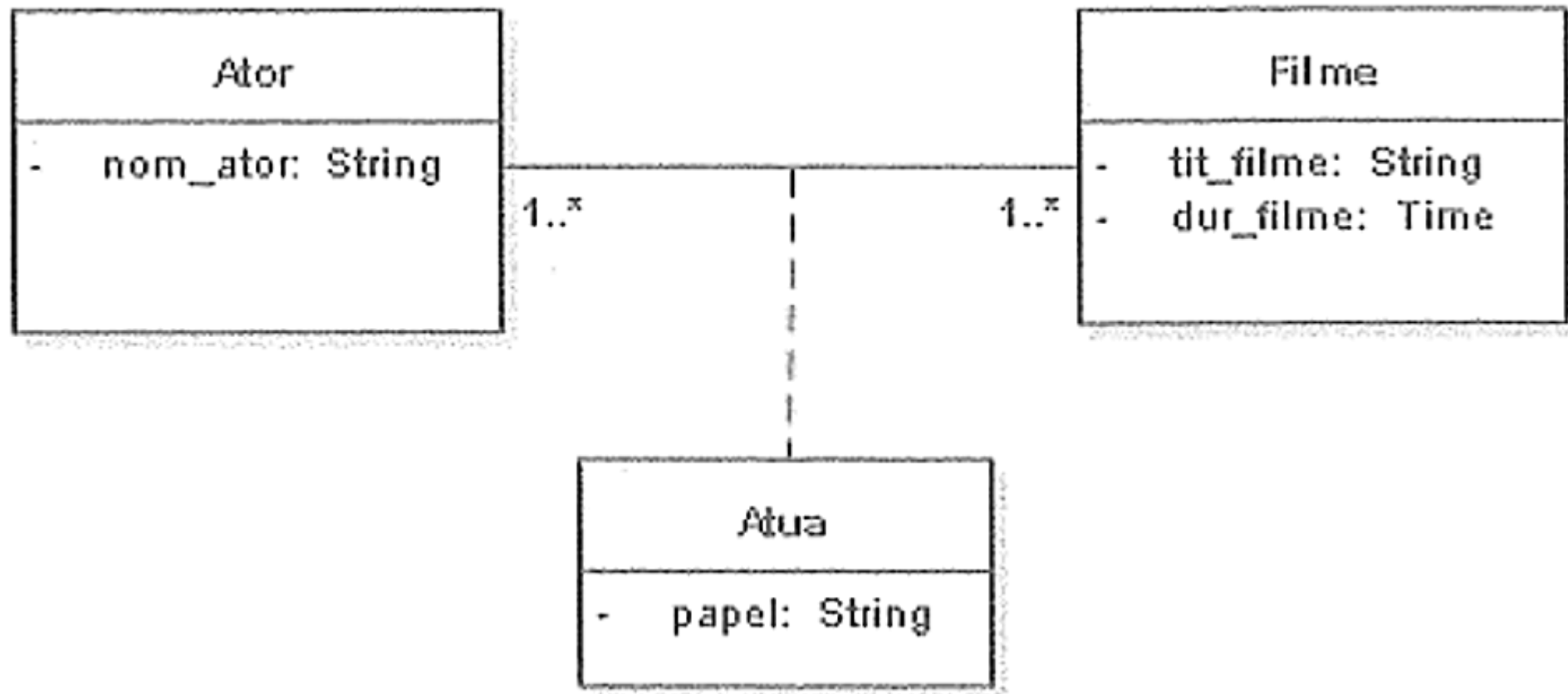
- Classe que está ligada a uma associação, em vez de estar ligada a outras classes
- Normalmente necessária quando duas ou mais classes estão associadas, e é necessário manter informações sobre esta associação
- Uma classe associativa pode estar ligada a associações de qualquer tipo de conectividade
- **Sinônimo:** classe de associação

Notação para Classes Associativas

- Notação é semelhante à utilizada para classes ordinárias
- A diferença é que esta classe é ligada a uma associação por uma linha tracejada
- Exemplo: para cada par de objetos [pessoa, empresa], há duas informações associadas: salário e data de contratação



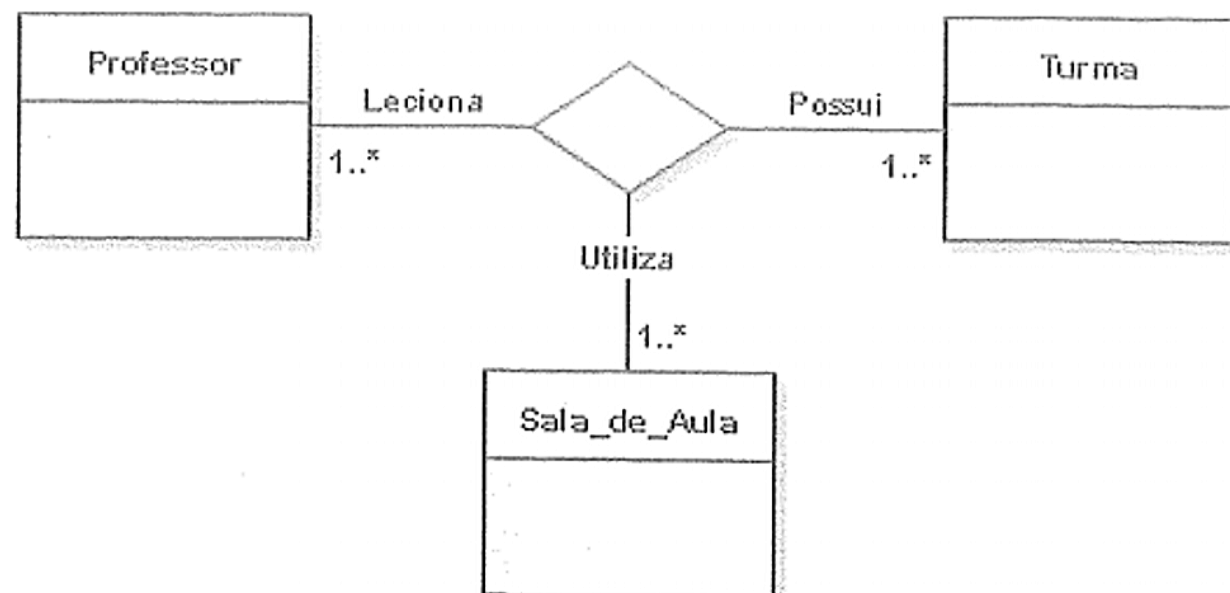
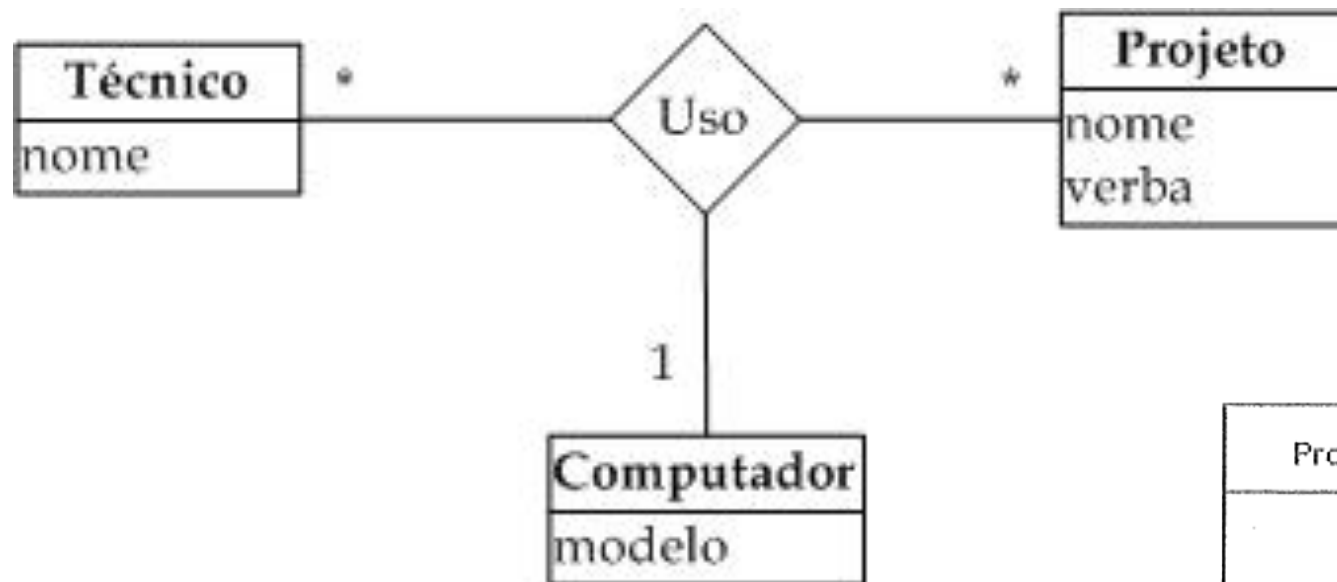
Notação para Classes Associativas



Associações n-árias

- Define-se o **grau** de uma associação como a quantidade de classes envolvidas na mesma
- Linhas de uma associação **n-ária** se interceptam em um losango
- Maioria dos casos práticos de modelagem, as associações normalmente são **binárias**
- Quando o grau de uma associação é igual a três, dizemos que a mesma é **ternária**
 - Uma associação ternária são uma caso mais comum (menos raro) de associação n-ária ($n = 3$)

Exemplo (associação ternária)

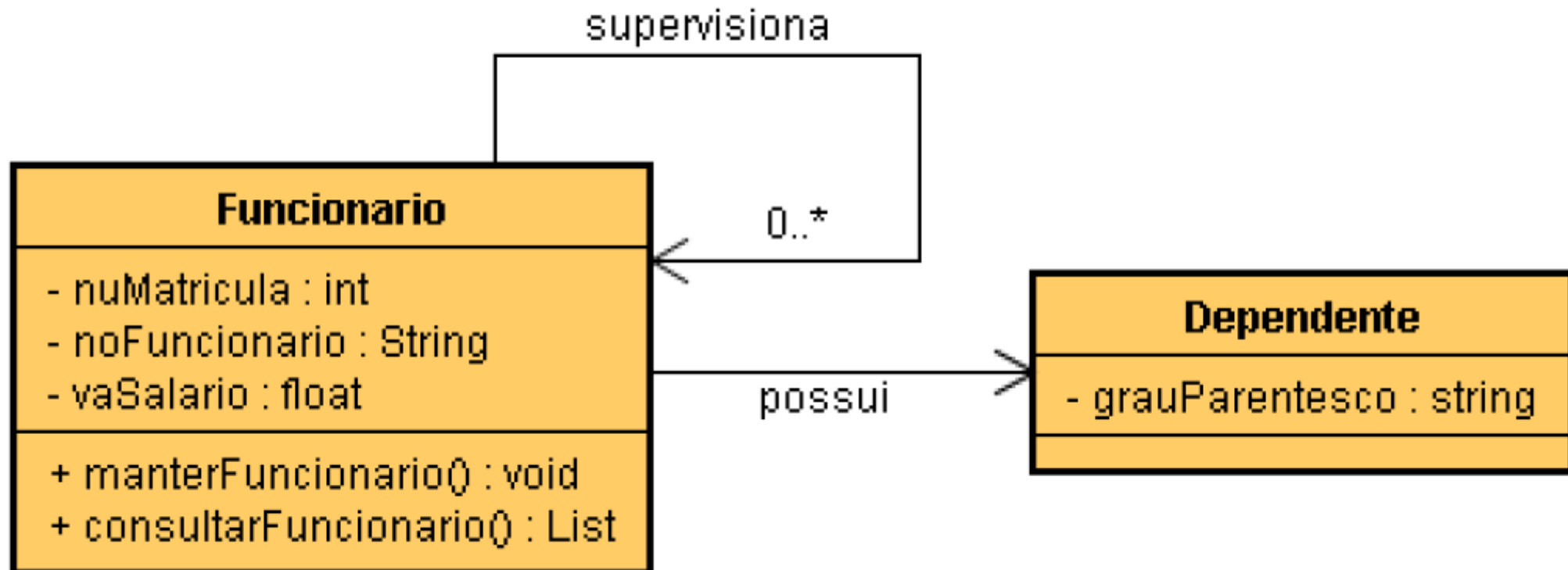


Associações reflexivas

- Tipo especial de associação que representa ligações entre objetos que pertencem a uma mesma classe
 - Não indica que um objeto se associa a ele próprio
- Quando se usa associações reflexivas, a definição de papéis é importante para evitar ambiguidades na leitura da associação
 - Cada objeto tem um papel distinto na associação



Associações reflexivas



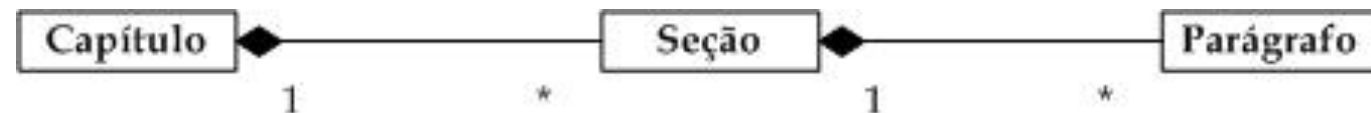
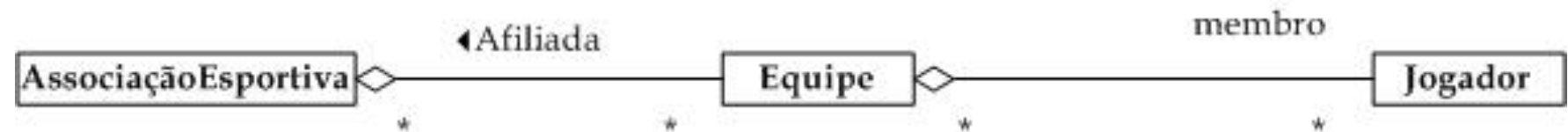
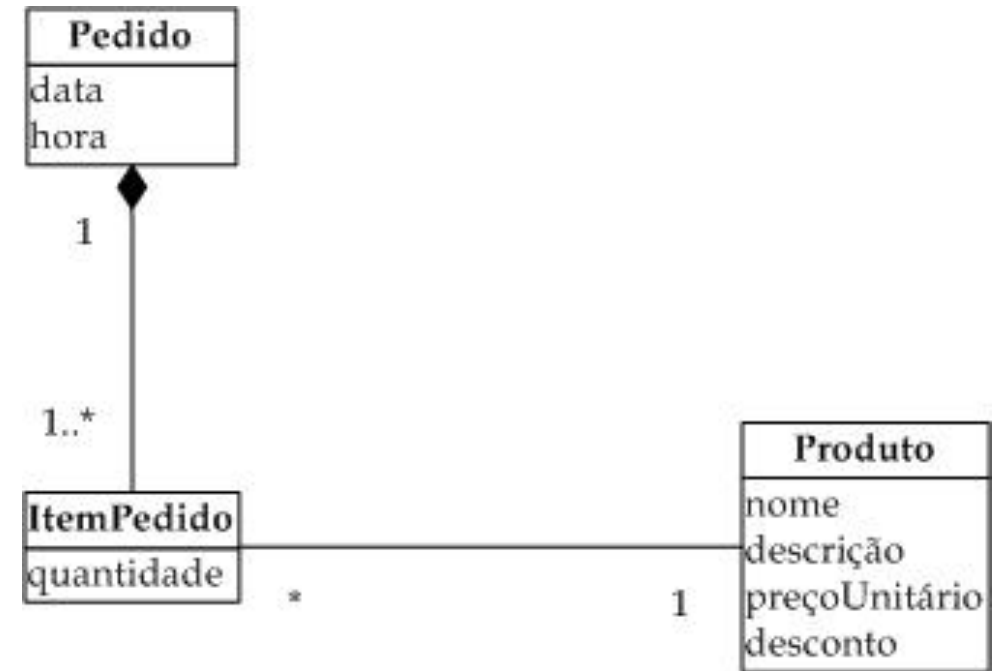
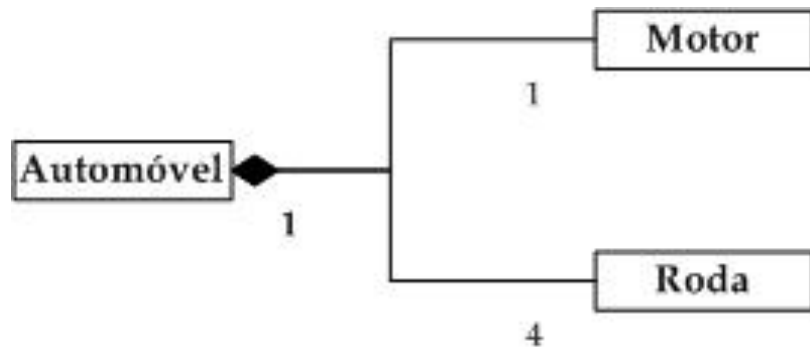
Agregações e Composições

- Semântica de uma associação corresponde ao seu significado
 - Natureza conceitual da relação que existe entre os objetos que participam daquela associação
- Relações todo-parte
 - Uma relação todo-parte entre dois objetos indica que um dos objetos está contido no outro
 - Um objeto contém o outro
- UML define dois tipos de relacionamentos todo-parte, a agregação e a composição

Agregações e Composições

- Particularidades das agregações/composições:
 - São assimétricas, no sentido de que, se um objeto A é parte de um objeto B, o objeto B não pode ser parte do objeto A
 - Propagam comportamento, no sentido de que um comportamento que se aplica a um todo automaticamente se aplica às suas partes
 - As partes são normalmente criadas e destruídas pelo todo. Na classe do objeto todo, são definidas operações para adicionar e remover as partes
- Se uma das perguntas a seguir for respondida com um sim, provavelmente há uma agregação onde X é todo e Y é parte
 - X tem um ou mais Y?
 - Y é parte de X?

Exemplos

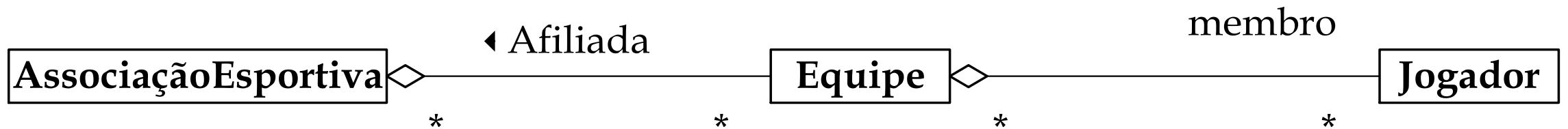


Agregações e Composições – Diferenças

- Destruição de objetos
 - Na agregação, a destruição de um objeto todo não implica necessariamente na destruição do objeto parte
- Pertinência
 - Na composição, os objetos parte pertencem a um único todo
 - Por essa razão, a composição é também denominada agregação não-compartilhada
 - Em uma agregação, pode ser que um mesmo objeto participe como componente de vários outros objetos
 - Por essa razão, a agregação é também denominada agregação compartilhada

Notação para Agregação

- Linha que conecta as classes relacionadas, com um diamante (losango) branco perto da classe que representa o todo



Notação para Composição

- Linha que conecta as classes relacionadas, com um diamante (losango) negro perto da classe que representa o todo



Agregação

- Associação em que um objeto é parte de outro, de tal forma que a parte pode existir sem o todo



```
public class A {
    private B b;
    public A( ){
    }
    public void setB( B b ){
        this.b = b;
    }
    public B getB( ) {
        return b;
    }
}

public class B {
    public B( ){
    }
}
```


Composição

- O todo contém as partes (e não referências para as partes)
- Quando o todo desaparece, todas as partes também desaparecem



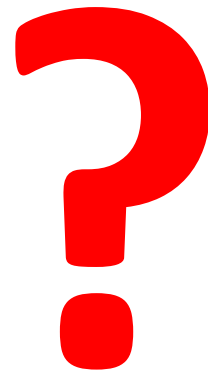
```
public class A {
    private B b;
    public A( ){
        b = new B();
    }
}

public class B {
    public B( ){
    }
}
```

Agregações vs Composições

- Diferenças são tênues e essas são as mais visíveis:
 - Agregação
 - Destruição de um objeto todo não implica necessariamente a destruição do objeto parte
 - Composição
 - Objetos parte pertencem a único objeto todo, por isso quando um objeto todo é destruído os objetos parte também são

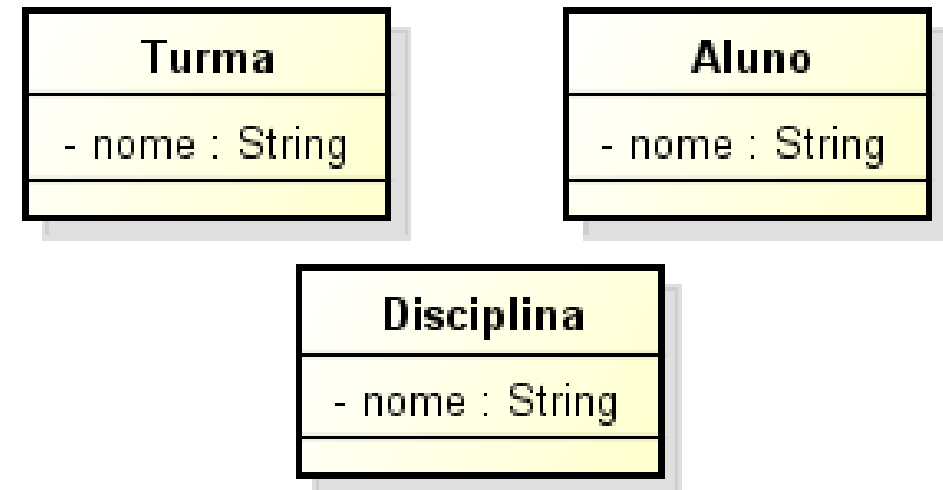
Dúvidas





Exercícios

- Considere as 3 classes
- Contexto de um sistema de controle acadêmico
- Crie e explique relacionamentos de agregação e composição



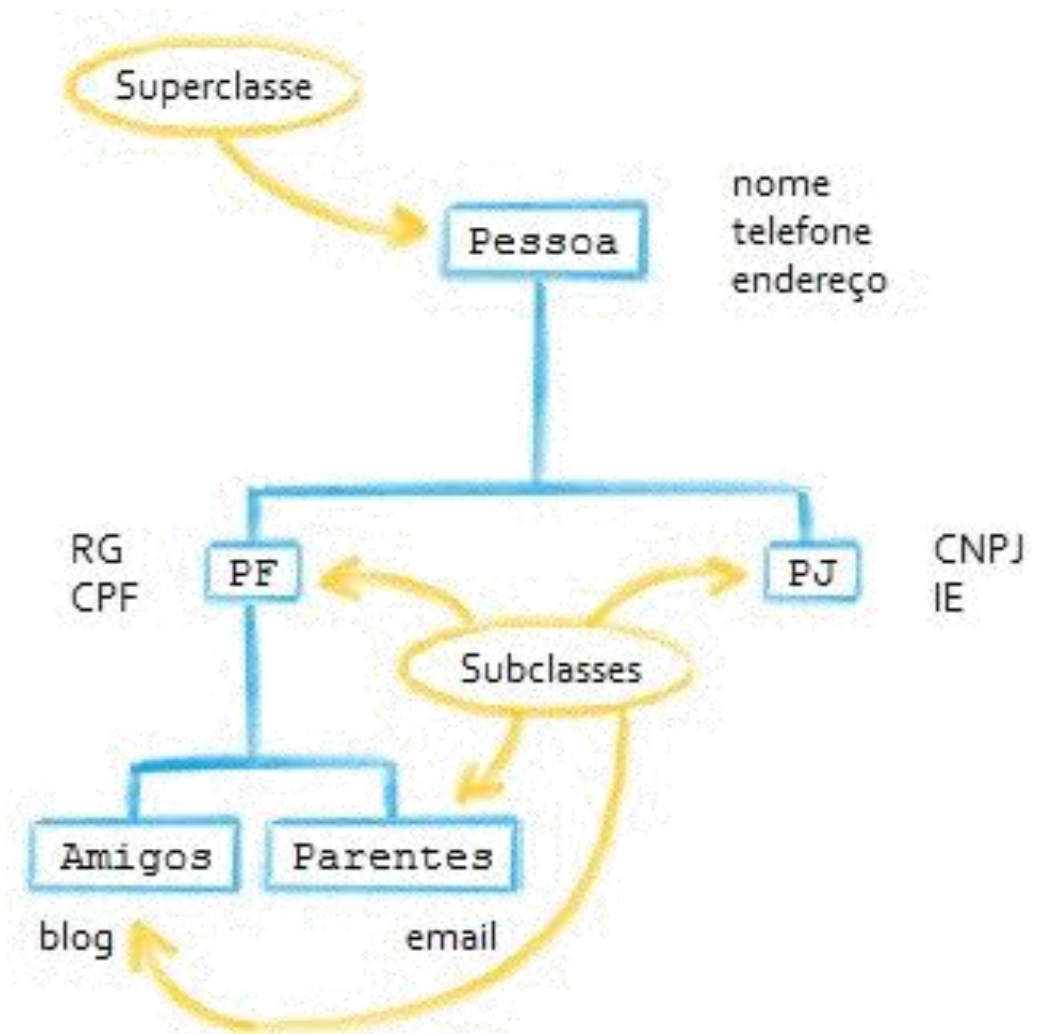
Herança

Herança

- Mecanismo para estender as classes existentes adicionando novos métodos e campos
 - Aprimora classes existentes
 - Inclui ou modifica métodos
 - Inclui novas propriedades (campos)
- Herdar é derivar características de gerações precedentes
 - Reutilização de software

Herança

- Classe derivada herda da classe básica:
 - Atributos
 - Comportamentos



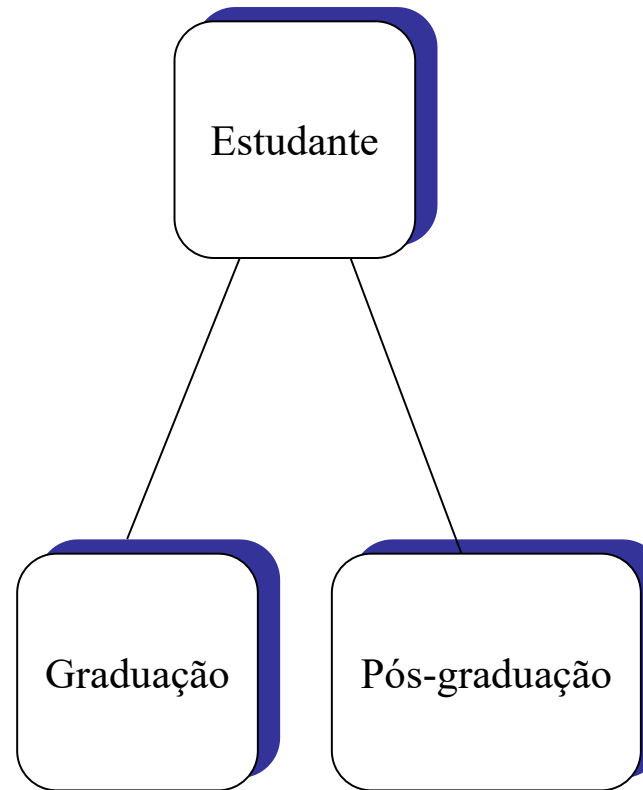
Especialização / Generalização

O que é GENERALIZAÇÃO ?

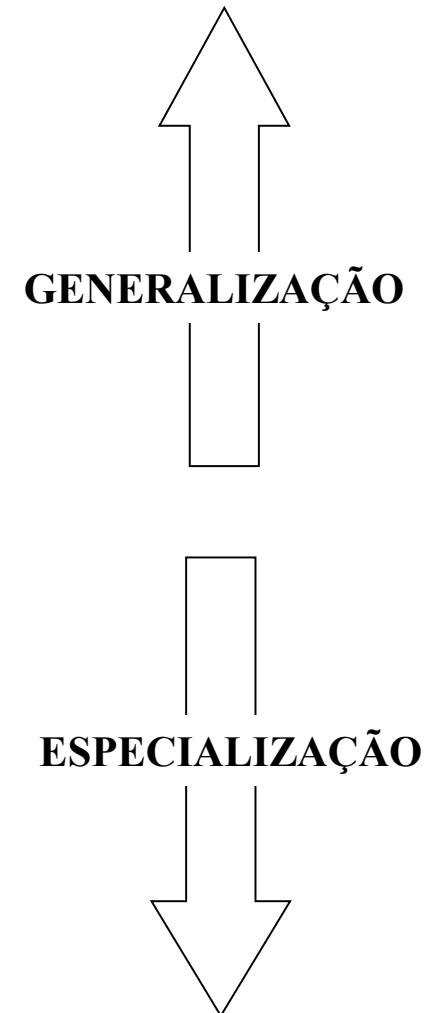
O que é ESPECIALIZAÇÃO ?

Especialização / Generalização

- Classe básica
 - Mais genérica
- Classes derivadas
 - Mais especializada

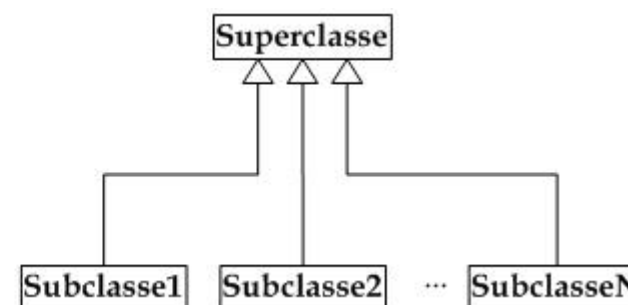
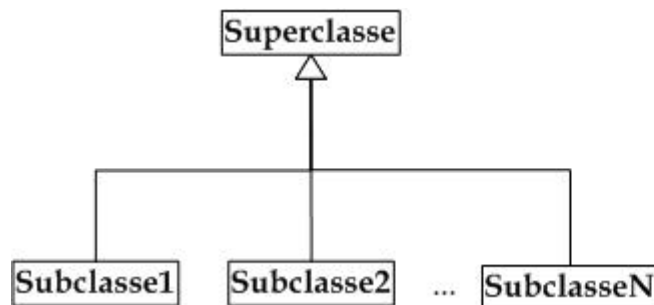


HIERAQUIA DE CLASSES



Generalizações e Especializações

- Terminologia
 - subclasse X superclasse
 - supertipo X subtipo
 - classe base X classe herdeira
 - classe de especialização X classe de generalização
 - ancestral e descendente (herança em vários níveis)
- Notação definida pela UML

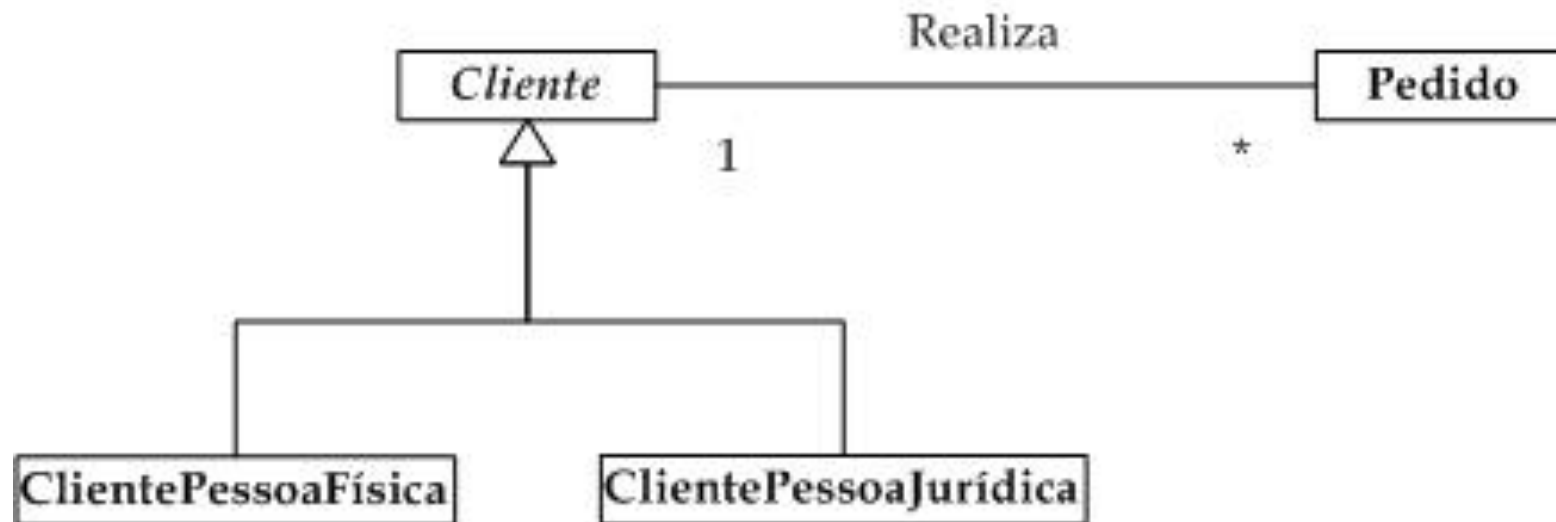


Semântica da Herança

- Subclasses herdam as características de sua superclasse
 - É como se as características da superclasse estivessem definidas também nas suas subclasses
 - Essa herança é transitiva e anti-simétrica
- Diferença semântica entre a herança e a associação
 - Herança trata de um relacionamento entre classes, associação representa relacionamentos entre instâncias de classes
 - Na associação, objetos específicos de uma classe se associam entre si ou com objetos específicos de outras classes
 - Exemplo:
 - Herança: “Gerentes são tipos especiais de funcionários”
 - Associação: “Gerentes chefiam departamentos”

Herança de Associações

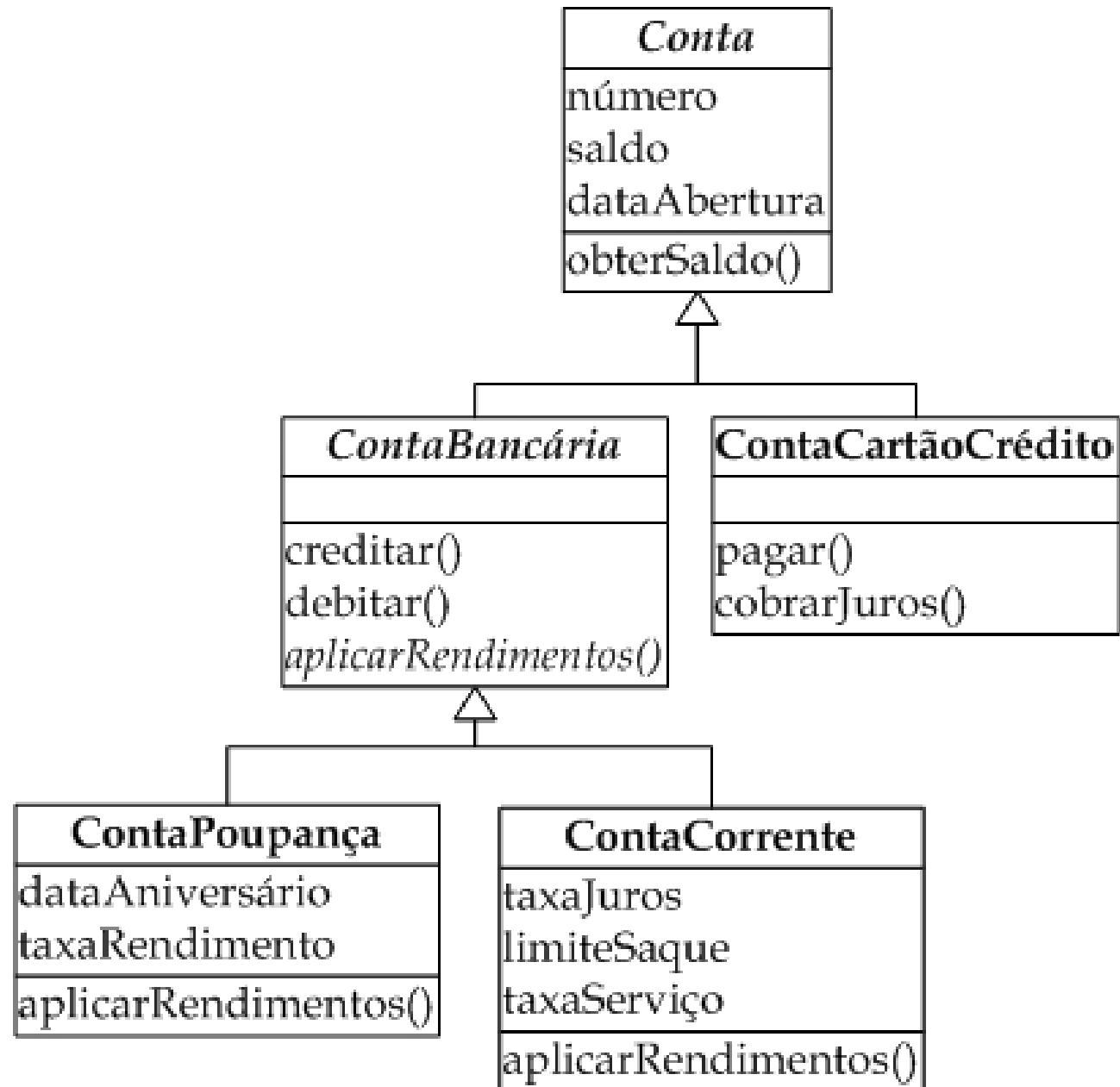
- Não somente atributos e operações, mas também associações são herdadas pelas subclasses
- Ex: cada subclasse está associada a Pedido, por herança



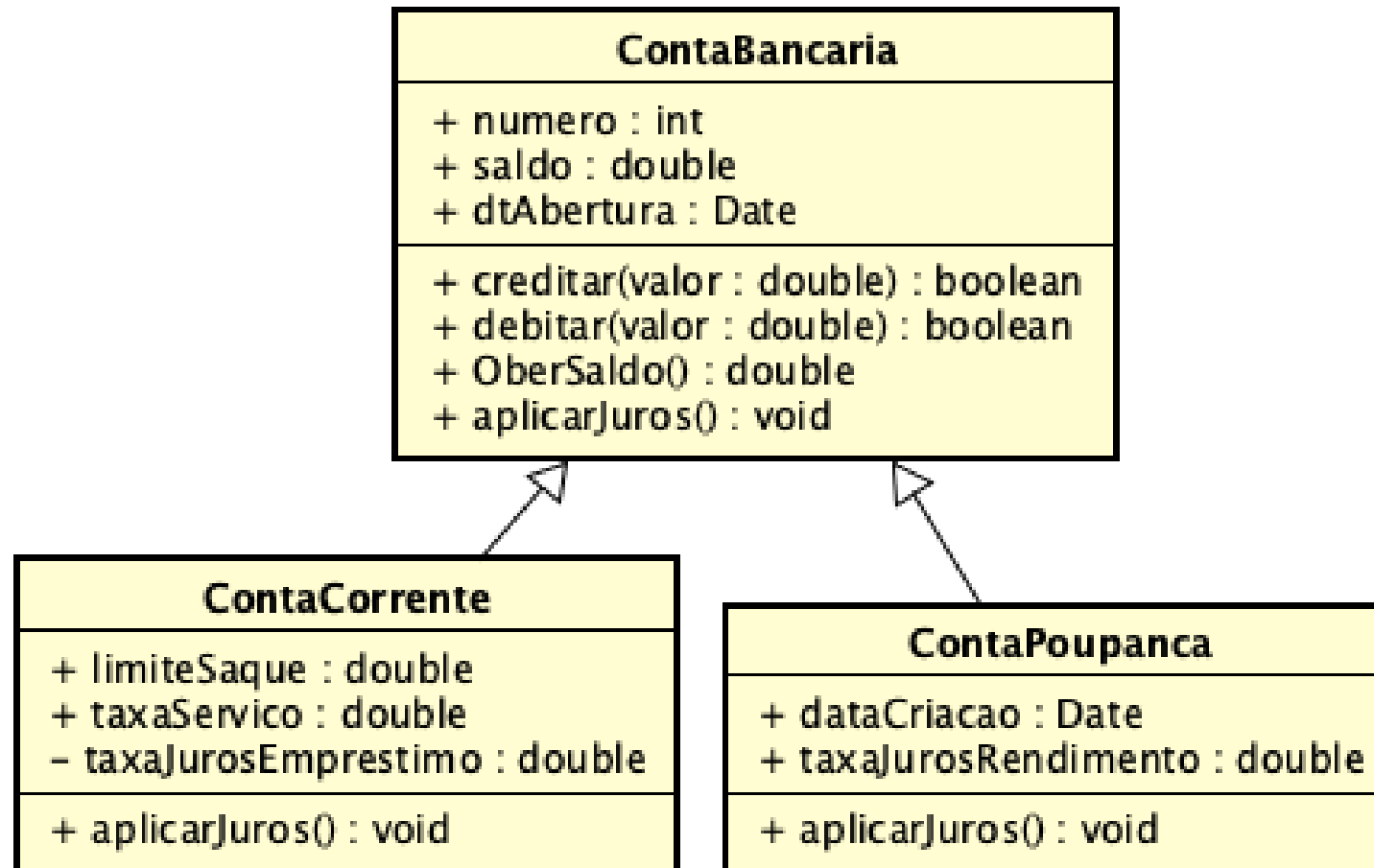
Propriedades da Herança

- **Transitividade:** uma classe em uma hierarquia herda propriedades e relacionamentos de todos os seus ancestrais
 - Herança pode ser aplicada em vários níveis, dando origem a hierarquia de generalização
 - Uma classe que herda propriedades de uma outra classe pode ela própria servir como superclasse
- **Assimetria:** dadas duas classes A e B, se A for uma generalização de B, então B não pode ser uma generalização de A
 - Não pode haver ciclos em uma hierarquia de generalização

Propriedades da Herança



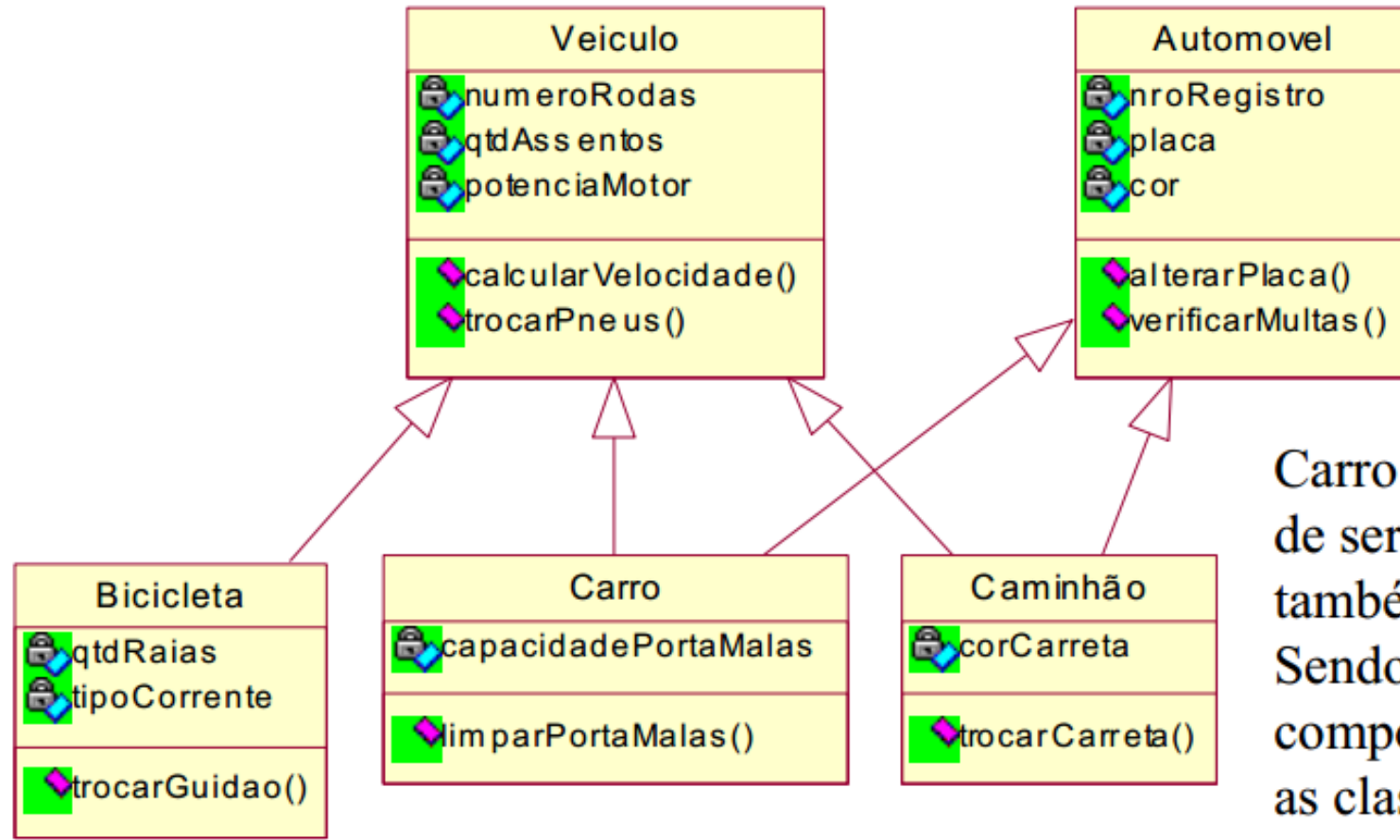
Propriedades da Herança



Tipos de Herança

- Quantidade de superclasses que certa classe pode ter:
 - Herança simples
 - Herança múltipla

Herança Múltipla

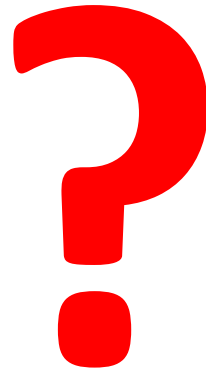


Carro e Caminhão, além de serem Veículos também são automóveis. Sendo assim, herdam o comportamento de ambas as classes

Refinamento do Modelo com Herança

- Critérios a avaliar na criação de subclasses:
 - A subclasse tem atributos adicionais
 - A subclasse tem associações
 - A subclasse é manipulada (ou reage) de forma diferente da superclasse
- Se algum “subconceito” (subconjunto de objetos) atenda a alguns dos critérios acima, a criação de uma subclasses deve ser considerada
- Sempre se assegure de que se trata de um relacionamento do tipo “é-um”: X é um tipo de Y?

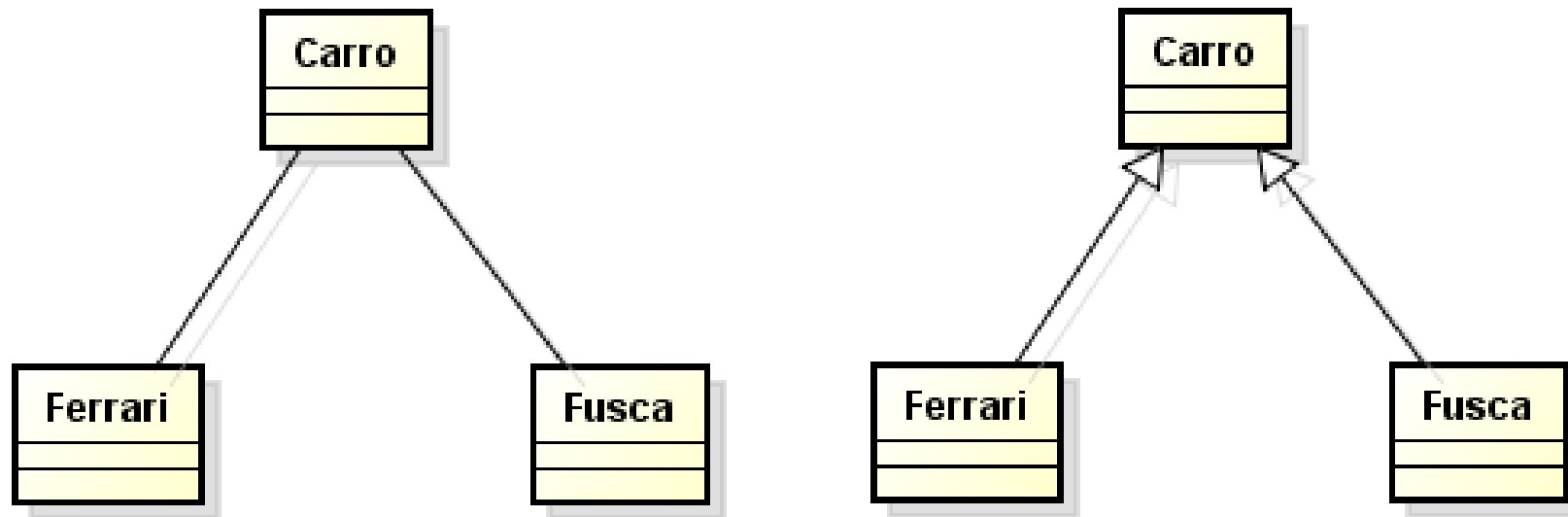
Dúvidas



Exercícios



- Qual a diferença entre os dois diagramas abaixo ?





Exercícios

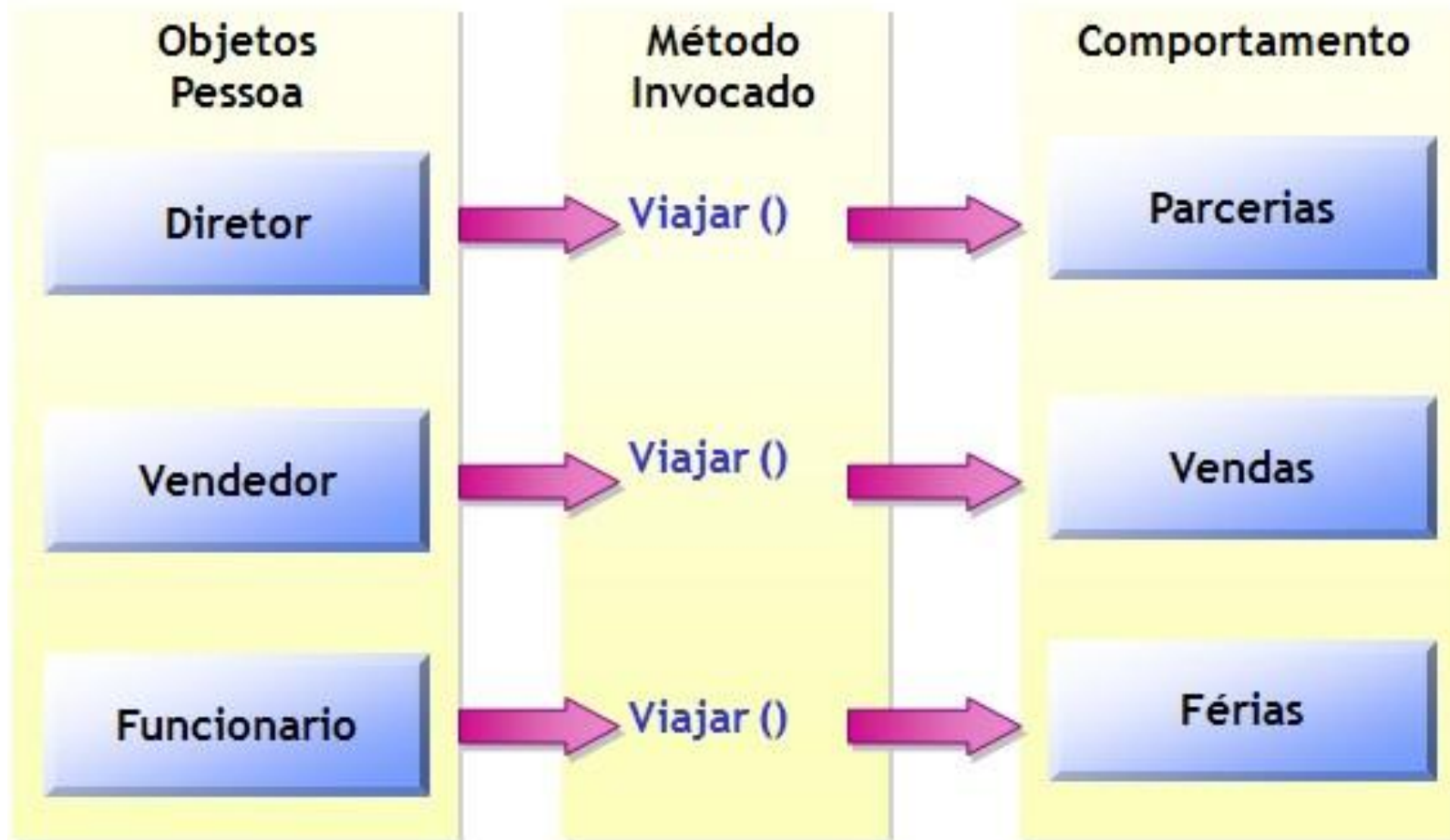
- Modele uma relação de herança para um sistema de controle acadêmico
- Considere diversos perfis, classes pai e classes filhas

Polimorfismo

Polimorfismo

- “Muitas Formas”
- Representa o fato de uma determinada característica pode ser diferente para cada filho
- Termo definido em linguagens OO para a possibilidade de se usar o mesmo elemento de forma diferente
- Parte de um objeto mais simples que vai evoluindo
- Conceito do objeto pai continua a existir
 - Mesmo que tenham sofrido modificações ou assumido novas formas

Polimorfismo



Polimorfismo

- Diferentes tipos de objetos podem responder a uma mesma mensagem de maneiras diferentes

Método Imprime() [Documento → cheque, fotografia;]

- Dizemos que imprime() é um método polimórfico pois ele é implementado diferentemente por diferentes tipos de objetos

Formas de Polimorfismo

- Redefinição (overriding)
 - Mesma função mas comportamentos diferentes
 - Ex:
 - Operação movimentar() para mamífero e para peixe
 - Operação minimizarJanela() para Windows e para Linux
- Sobrecarga (overloading)
 - Nome de uma função usado mais de uma vez com diferentes tipos de parâmetros
 - Ex:
 - Operação soma() com dois parâmetros reais e outra com parâmetros inteiros

Sobrecarga de método - Overloading

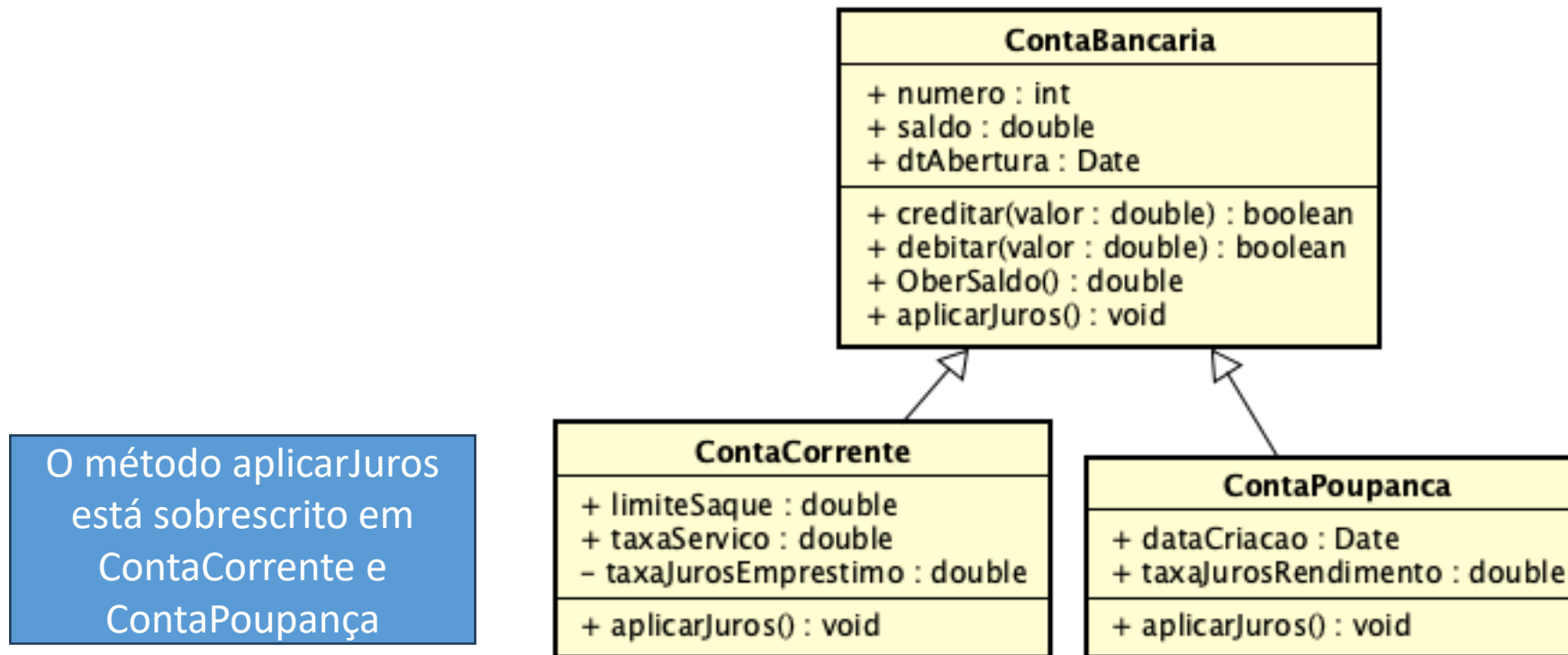
- Uma classe **sobrecarrega** um método quando disponibiliza dois métodos com mesmo nome, porém com parâmetros diferentes em quantidade, ordem ou tipo

ContaCorrente
+ limiteSaque : double + taxaServico : double + taxaJurosEmprestimo : double
+ contaCorrente(limiteSaque : double, taxaServico : double, taxaJurosEmprestimo : int) + transferencia(valor : double, agencia : int, numero : int) : boolean + transferencia(valor : double, agencia : int, numero : int, variacaopoupanca : int) : boolean

O método transferência
está sobrecarregado em
ContaCorrente

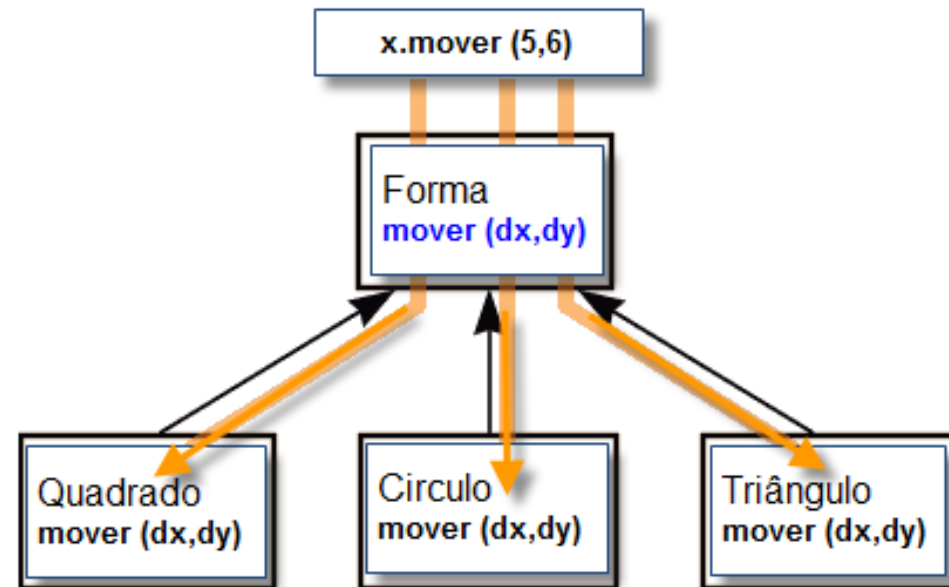
Sobrescrita de método - Overriding

- Uma classe **sobrescreve** um método quando herda um método e define um método com o mesmo nome e parâmetros do método herdado da classe pai



Polimorfismo – Exemplo – Overriding

```
Animal a;  
a = new Cachorro();  
a.falar(); → au! au!  
a = new Gato();  
a.falar(); → miau! miau!  
a = new Galinha();  
a.falar(); → cocorocó!
```



Classes Abstratas e Interfaces

Motivação

Como você obrigaria uma pessoa a implementar algo do jeito que foi definido?

Como você padronizaria um conjunto de classes semelhantes?

Motivação

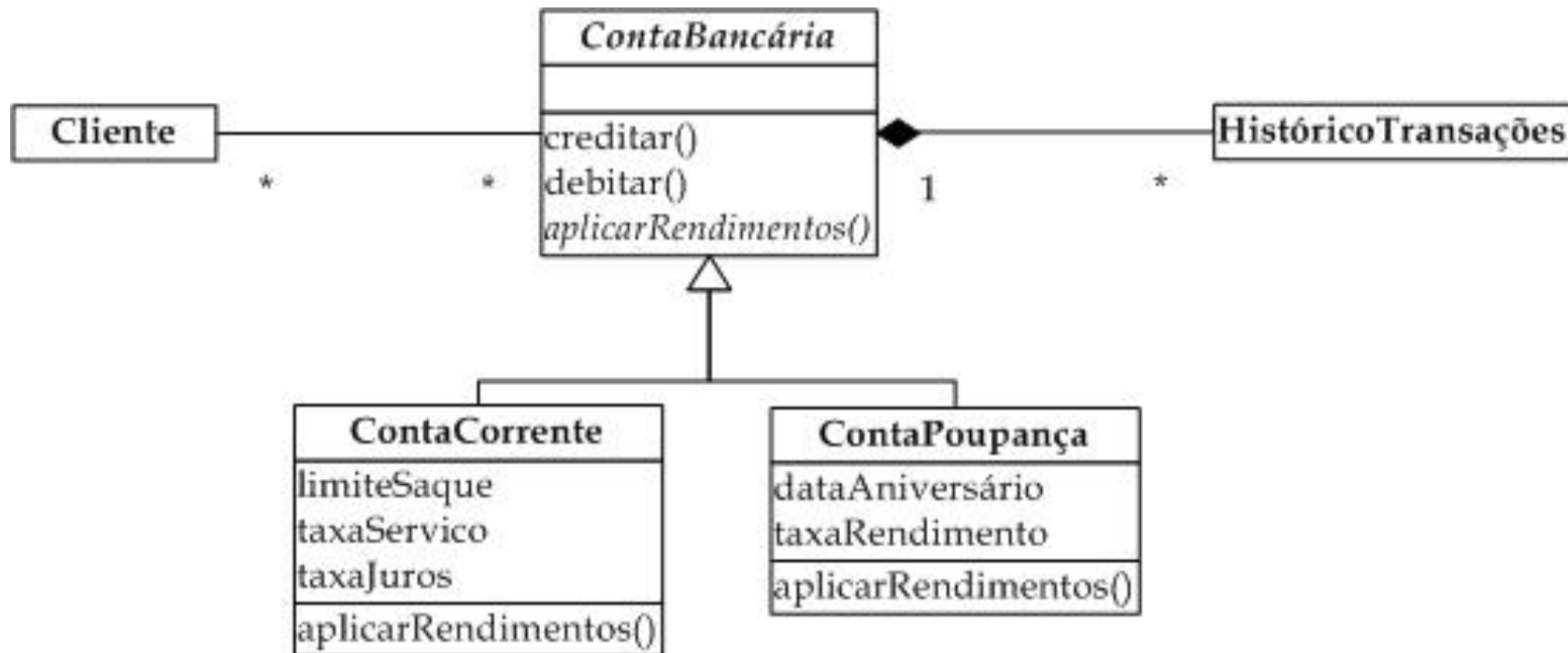
- Utilização de herança de classes e sobreposição
 - Muitas vezes não é solução ideal
- Necessidade de criar uma superclasse que defina uma forma generalizada
 - Compartilhada por todas suas subclasses
 - Implementação dos detalhes fica para essas subclasses
- Superclasse
 - Define apenas o esqueleto dos métodos
 - Subclasses devem implementar

Classes Abstratas

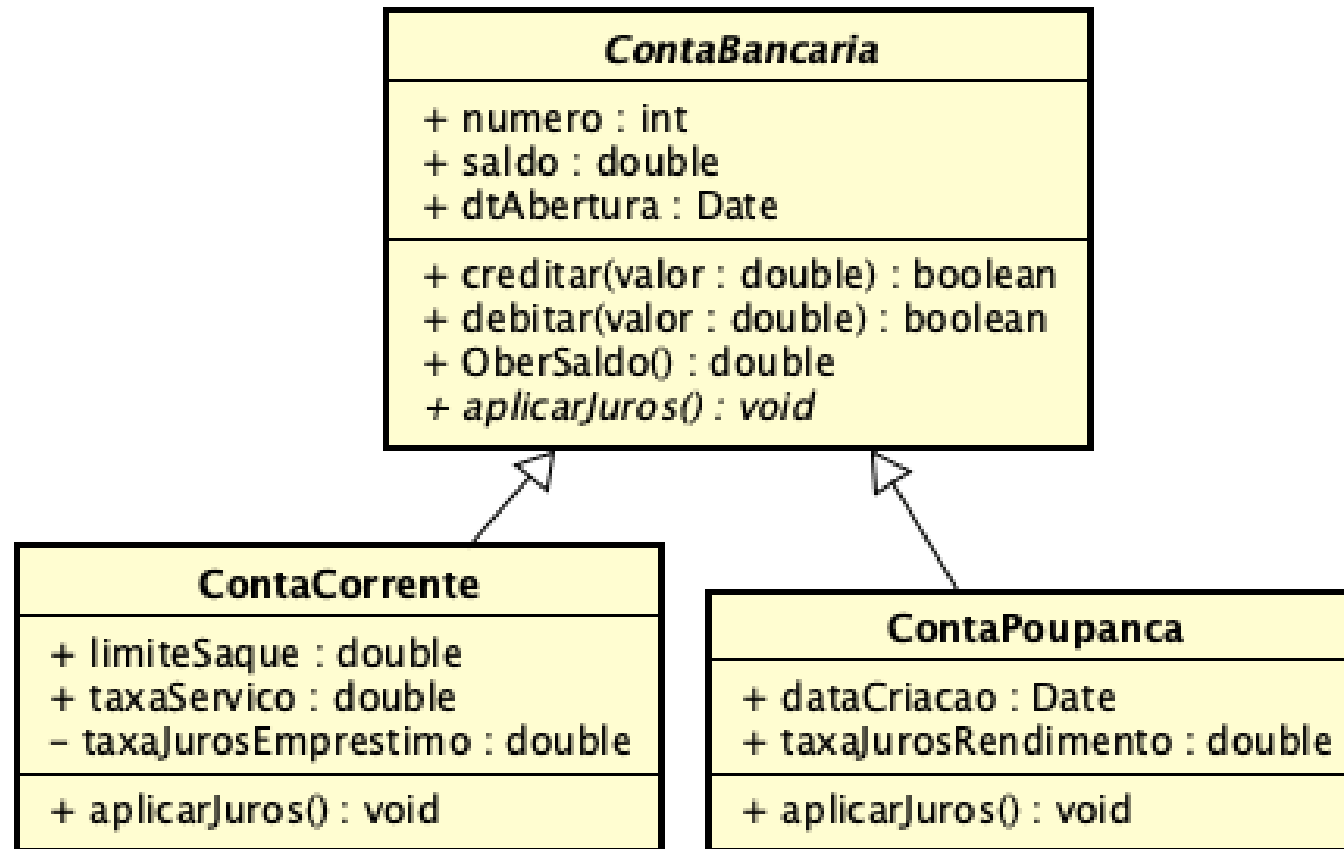
- Usualmente, a existência de uma classe se justifica pelo fato de haver a possibilidade de gerar instâncias da mesma
 - Essas são as classes **concretas**
- No entanto, podem existir classes que não geram instâncias diretas
 - Essas são as classes **abstratas**
- Classes abstratas
 - Utilizadas para organizar e simplificar uma hierarquia de generalização
 - Propriedades comuns a diversas classes podem ser organizadas e definidas em uma classe abstrata a partir da qual as primeiras herdam
- Subclasses de uma classe abstrata também podem ser abstratas, mas a hierarquia deve terminar em uma ou mais classes concretas

Notação para classes abstratas

- Representada com o seu nome em itálico
- Ex: ContaBancária é uma classe abstrata



Notação para classes abstratas



Operações Abstratas

- Uma classe abstrata **possui ao menos uma operação abstrata**, que corresponde à especificação de um serviço que a classe deve fornecer (sem método)
- Quando uma subclasse herda uma operação abstrata e não fornece uma implementação para a mesma, esta classe também é abstrata

```
1 abstract class Tudo {  
2     String nome;  
3  
4     String retornaNome() {  
5         return nome;  
6     }  
7  
8     abstract void comunicar();  
9 }
```

```
1 class Animal extends Tudo {  
2     int qtdPatas;  
3     String sobrenome;  
4     public void comunicar() {  
5         println("o animal genérico está se comunicando");  
6     }  
7     public void andar() {  
8         println("o animal genérico está andando");  
9     }  
10    public void comer() {  
11        println("o animal genérico está comendo");  
12    }  
13    public void dormir() {  
14        println("o animal genérico está dormindo");  
15    }  
16    public void outraAcao() {  
17        println("o animal genérico fez qualquer outra coisa");  
18    }  
19 }
```

Interfaces

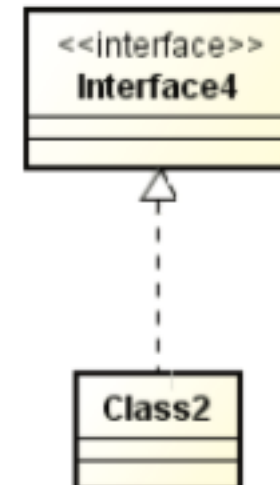
- Uma interface entre dois objetos compreende um **conjunto de assinaturas de operações** correspondentes aos serviços dos quais a classe do objeto cliente faz uso
- Uma interface pode ser interpretada como um **contrato** de comportamento entre um objeto cliente e eventuais objetos fornecedores de um determinado serviço
- Tipo especial de classe a qual não pode ser instanciada
 - Serve apenas para especificar operações externamente visíveis para uma outra classe implementar

Interface

- Interface = Interface de código

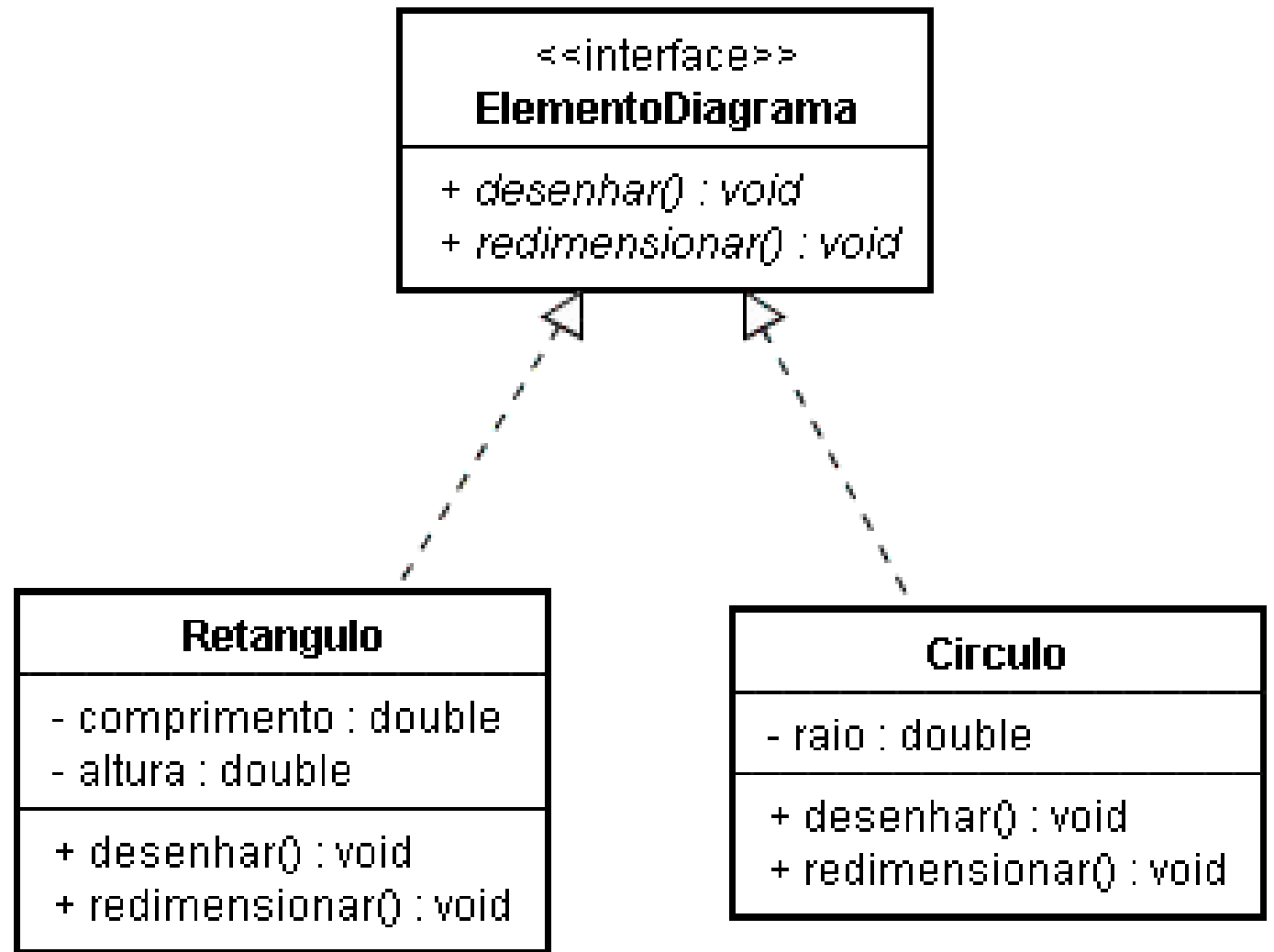


- Realização = generalização/especialização entre interface e classe



Interfaces

O relacionamento de realização foi utilizado aqui: É uma herança na qual as entidades específicas implementam os métodos abstratos




```
1 abstract class Tudo {  
2     String nome;  
3  
4     String retornaNome() {  
5         return nome;  
6     }  
7  
8     abstract void comunicar();  
9 }
```

```
1 interface Acoes {  
2     void andar();  
3     void comer();  
4     void dormir();  
5 }
```

```
1 class Animal extends Tudo implements Acoes {  
2     int qtdPatas;  
3     String sobrenome;  
4     public void comunicar() {  
5         println("o animal genérico está se comunicando");  
6     }  
7     public void andar() {  
8         println("o animal genérico está andando");  
9     }  
10    public void comer() {  
11        println("o animal genérico está comendo");  
12    }  
13    public void dormir() {  
14        println("o animal genérico está dormindo");  
15    }  
16    public void outraAcao() {  
17        println("o animal genérico fez qualquer outra coisa");  
18    }  
19 }
```

Dependência

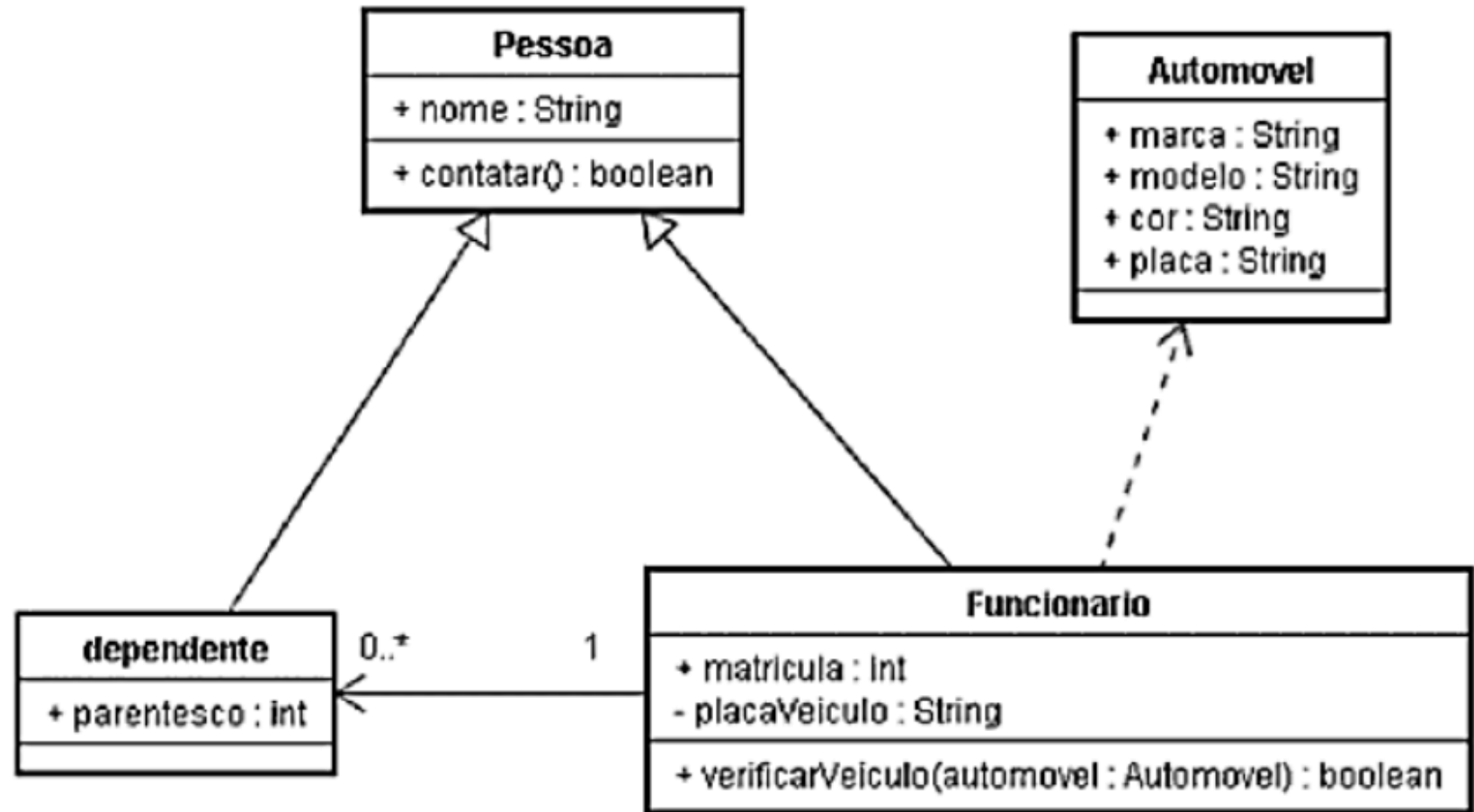
- Indica que uma classe depende dos serviços (operações) fornecidos por outra classe
- Entretanto, há também as dependências não estruturais
 - Dependência por variável global
 - Um objeto de escopo global é referenciado em algum método da classe dependente
 - Dependência por variável local
 - Um objeto recebe outro como retorno de um método, ou possui uma referência para o outro objeto como uma variável local em algum método
 - Dependência por parâmetro
 - Um objeto recebe outro como parâmetro em um método

Dependência

- Dependências não estruturais são representadas na UML por uma linha tracejada direcionada e ligando as classes envolvidas
 - A direção é da classe dependente (cliente) para a classe da qual ela depende (fornecedora)
 - Indica que uma classe usa informações de outra

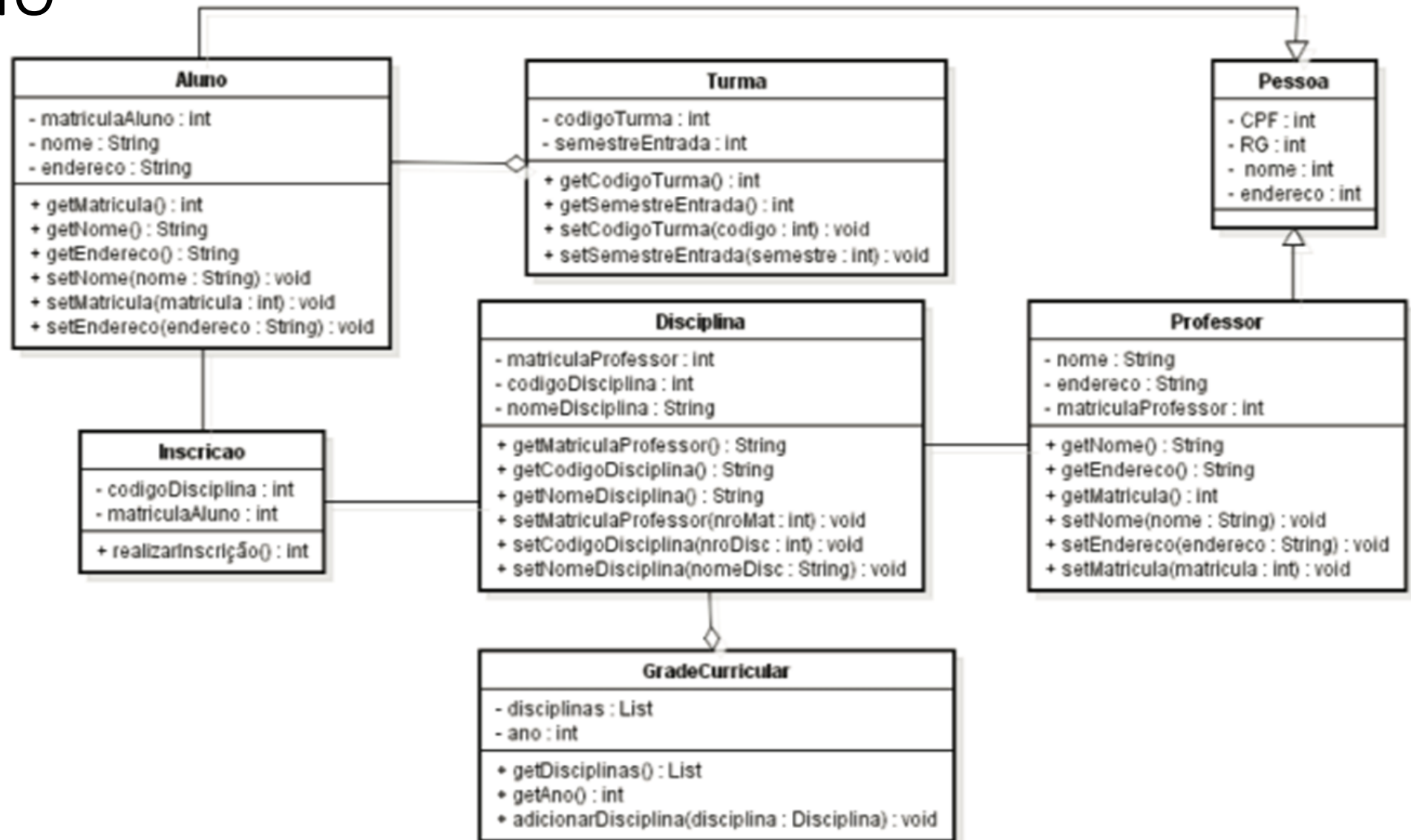


Exemplo

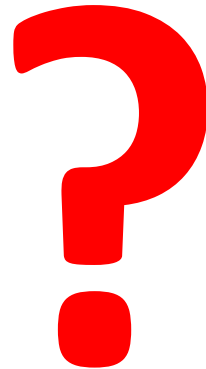


Funcionário não instancia um Automóvel, apenas usa-o como parâmetro de um método.

Exemplo



Dúvidas



Referências Bibliográficas

- Google Imagens
- Freepik
- Análise e Projeto de Sistemas - Universidade Federal do Ceará – UFC
- Campus de Quixadá - Curso de Sistemas de Informação - Prof. Enyo José
- Entendendo a Programação Orientada a Objetos com Python - <https://medium.com/@guilhermerdcarvalho/paradigma-orientado-ao-objeto-poo-em-python-a107d35bee3f>
- Análise e Projeto de Sistemas – OO – Emanuel Coutinho
- PROJETO DE DESENVOLVIMENTO DE SOFTWARE - Professor: Diego Oliveira - Aula 12: Diagrama de Classes
- Engenharia de Software - Diagrama de Classe - Givanaldo Rocha de Souza – IFRN